ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

On Combiners for Verifiable Random Functions

Bachelor Thesis

Basil Rohner

March 17, 2026

Advisors: Prof. Dr. D. Hofheinz, N. Brandt, X. Duan
Department of Computer Science, ETH Zürich

Acknowledgments

I would like to thank my supervisors Nicholas Brandt and Xiaoqi Duan for their continuous support and patience. I also want to thank Professor Dennis Hofheinz for the opportunity to write my thesis at his group. Finally, I would like to thank my family and friends for their endless support and encouragement.

"The wise man does at once what the fool does finally."

— Niccolò Machiavelli

Abstract

Verifiable random functions (VRFs) are an extension of pseudorandom functions (PRFs), one can evaluate the function using a secret key to obtain random-looking images and corresponding proofs. Given the corresponding public key one can verify whether a given image was generated from an input if given a suitable proof.

In this thesis we study ways to combine multiple instances of VRFs in a black-box fashion. Specifically we study the existence of robust black-box VRF-combiners, constructions which are given oracle access to a set of candidate schemes, of which a sufficiently large (unknown) subest implements the VRF securely, and implements a secure VRF.

While there exist simple constructions of such robust black-box combiners for the related PRF primitive even if only one candidate implements a secure PRF, we show that there exists no robust black-box VRF-combiners if not at least half of the candidate schemes are secure, even if we relax the required security properties from the combiner.

Contents

Contents **iv**

1 Introduction **1**
1.1 Motivation and Previous Work 1
1.2 PRF Combiners 3
1.3 Our Contribution 4

2 Notation and Preliminaries **7**
2.1 Notation 7
2.2 Verifiable Functions 8
2.3 Robust Combiners 11

3 Results **13**
3.1 Robust Black-Box DAG Combiner 13
3.2 Impossibility of Robust Black-Box VRF-Combiner 18
3.3 Relaxed Robust Black-Box VRF-Combiners 38

4 Conclusion **41**

Bibliography **42**

Appendix **43**
Proof of Lemma .1 43
Conjecture 43
Proof of Claim 3.22 44

Chapter 1

Introduction

1.1 Motivation and Previous Work

Pseudorandom Functions. A pseudorandom function (PRF) [GGM86] is a deterministic algorithm that takes as input a secret key k and a point x to produce an output y that looks random in the sense that no efficient adversary, modeled by a probabilistic polynomial time (ppt.) Turing machine can distringuish the output y from a random bitstring r of the same length. This must hold even if the adversary has access to a polynoial number of point evaluations (x, y) before distinguishing the output. PRFs are fundamental to modern cryptography and used to construct, among other, symmetric encryption schemes [KL14], message authentication codes [BCK96], key derivation functions [Kra10].

Verifiable Functions. A verifiable function (VF) is a public-key primitive that allows the holder of a secret key sk to evaluate points x to obtain image-proof pairs (y, π) such that any party holding the public key pk can verify that y was indeed computed correctly from x if given the proof π. We require correctness and unique provability from the verification which state that correctly generated image-proof pairs (y, π) always (or except with negligible probability) is accepted by the verification algorithm and that at most one image can be proven respectively.

In this work we primarily study extensions of VRs, primarily the verifiable random function (VRF) introduced by Micali, Rabin and Vadhan in [MRV99] that additionally requires the outputs y to be pseudorandom to efficient adversaries without access to the corresponding proof. Similar to the pseudorandomness definition for PRFs we require that no efficient adversary can distinguish the images y from random outputs r, even if the adversary gets to see polynomially many evaluations of x with their corresponding

proof. VRFs can be seen as a public-key analogoue of PRFs with verifiability requirements.

In practice VRFs are used to implement signatures with strong verifiability guarantees [**?**], resettable zero-knowledge proofs [**?**], lottery and e-cash systems [**?**] and they play a big role in distributed leader election protocols that runs many Proof-of-Stake blockchain systems [**?**].

Additionally we study VFs satisfying weaker notions of pseudorandomness such as weak pseudorandomness and unpredictability which give rise to verifiable unpredictable functions (VUFs) [**?**] and weak verifiable random functions (wVRFs) [**?**] respectively. VUFs additionally require that the evaluation of the VF on a fresh evaluation point cannot be predicted by efficient adversaries except with negligible probability and wVRFs require that pseudorandomness holds, but only on uniformly random evaluation points x, instead of a point chosen by the adversary.

add stronger notions cVRF, khVRF, aggVRF, unbiasVRF if time

Robust Combiners. In cryptography we often take a reductionist approach when constructing new cryptographic primitives, in general constructions satisfying computational security properties rely on cryptographic hardness assumptions. To hedge against the breakdown of cryptographic hardness assumptions, and thus the subsequence failure of the construction we may want to base the security of systems on multiple hardness assumptions. Robust combiners [**?**] are constructions that combine n candidate schemes, concrete instances of the same primitive, and implement a secure instance of that primitive if a large enough subset of those candidate is secure. The notion of combiners has gained traction with the rise of post-quantum and hybrid-cryptography areas

Stronger version of robust combiners are robust black-box combiners [**?**] which are only given query access to the candidate schemes as oracles, i.e. cannot inspect the Turing machine of the candidate scheme. The motivation of a robust black-box combiner is to have a combiner that can implement a secure scheme only with access to sub-schemes with the underlying code of the candidate schemes are not accessible to the combiner. We wish for the combiner to derive its security from the evaluation of the secure subset of oracles, not by making any modifications to the candidates or simply implementing a secure instance of the primitive himself. For that reason we additionally require robust black-box combiners to implement the secure instance relative to a PSPACE complete oracle, that is guarantee the security against adversaries with access to a PSPACE oracle.

Robust black-box combiners have been considered for a variety of cryptographic primitives such as oblivious transfer [**?**], non-interactive zero-knowledge proofs (NIZK) [**?**], non-interactive witness-indistinguishable arguments (NIWI) [**?**], indistinguishability obvuscation (iO) [**?**] and signature

schemes [**?**]. Additionally many folklore constructions exist for PRF, key exchange protocols . While there exist VRF constructions rely on PRFs and NIWIs, for both of which combiners have been studied, black-box combiners for VRFs have, to the best of our knowledge, not been studied.

add some

1.2 PRF Combiners

VRF can be considered as an extension of PRFs to the public-key setting with the additional requirements of verifiability. Simple robust black-box PRF combiners exist. One combiner works by evaluating the evaluation point x with every candidate scheme to obtain images y_1, y_2 and returning the XOR of the two images $y = y_1 \oplus y_2$. Pseudorandomness of the combined scheme can be shown with a reductionist approach. Specifically we show that given an adversary against the combined scheme, we can construct an adversary against every candidate scheme running in the same time with the same advantage. By contraposition, assuming that at least one candidate scheme is pseudorandom and there exists no efficient combiner with non-negligible advantage against the candidate we show that there exists no efficient adversary against the combined scheme with non-negligible advantage. We illustrate the construction of a robuts $(2;1)$ black-box PRF-combiner in Section 1.2.

$\mathsf{Eval}_{\mathsf{PRF}}^{\mathcal{O}_1,\mathcal{O}_2}(k,x)$

1 : $y_1 \leftarrow \mathcal{O}_1(k,x)$
2 : $y_2 \leftarrow \mathcal{O}_2(k,x)$
3 : **return** $y_1 \oplus y_2$

Figure 1.1: Robust $(2;1)$ Black-Box PRF-combiner.

We outline the reduction and refer to the appendix for the formal proof. The reduction against candidate scheme $\mathcal{O}_i$ relies on the pseudorandomness oracle $\mathcal{O}_{eval}$ and simulates the other candidate $\mathcal{O}_{2-i}$ himself to answer queries of the combiners adversary with $\mathcal{O}_{eval}(x) \oplus \mathcal{O}_{2-i}(k,x)$. The adversary against the combined scheme then guesses the hidden bit b, trying to distinguish the random world from the real world which the reduction then uses to guess its own hidden bit b'. If the hidden bit $b' = 0$ then the adversary $\mathcal{A}$ is simulated perfectly in the sense that the reduction wins the security experiment with the same advantage of $\mathcal{A}$ while for bit $b' = 1$ the answers to the queries of the adversary are still uniformly random bitstrings and therefore the reduction also wins with the same probability.

Note that the existance of a robust $(2;1)$ black-box PRF-combiner naturally leads to a robust $(n;1)$ black-box VRF-combiner for any natural number n by

iteratively applying the construction on the construction and a new candidate. Since there exists a robust $(n;1)$ black-box PRF-combiner one could hope for a similar construction to work for VRFs. This is however not the case, as we will show in **??** there exist no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner for VRFs.

1.3 Our Contribution

Robust Black-Box DAG Combiner. We introduce the notion of robust black-box DAG combiners, formally defined in Definition 3.1, DAG combiners for short. DAG combiners are black-box combiners which are defined by directed acyclic graphs (DAGs) for every implemented algorithm. The implemented algorithms are specified through computation that runs along the DAG starting from a designated source vertex and ending in a target vertex. In the execution vertices correspond to queries to implementation functions of candidate oracles on an input computed from all preceding vertices by a ppt. Turing machine (without oracle access to the candidates). DAG combiners make the structure of the black-box combiner more explicit by structuring it into interleaved oracle queries to the candidats and intermediate computation, that we model with Turing machines without oracle access to the candidates. We illustrate this concept in Figure 1.2 on the example of the simple robust black-box PRF-combiner in Section 1.2.

To connect DAG combiners and general black-box combiners we show that the notions are indeed equivalent, that is, that for every general black-box combiner we can construct a DAG combiner that behaves identically in the sense that the algorithms defined by the combiners are equivalent. We state this result formally in Theorem 3.2. The concept of a robust black-box DAG combiner allows us to reason over a more structured representation of robust black-box combiners, more similar to a circuit in structure simplifying the proofs later on.

Robust Black-Box VRF-Combiner. Following the introduction of a robust black-box DAG combiner, we study the simplest robust black-box VRF-combiner, robust $(2;1)$ black-box VRF-combiners. We answer the question on whether there is a construction that combines two candidates, of which only at least one is secure, negatively. Specifically we show impossibility of robust $(2;1)$ black-box VRF-combiners in **??**.

Since we can naturally use a robust $(n;\lfloor n/2 \rfloor)$ black-box VRF-combiner to construct a $(2;1)$ combiner by simply initializing half the candidates with $\mathcal{O}_1$ and the other half with $\mathcal{O}_2$ we get an impossibility result for robust $(n;\lfloor n/2 \rfloor)$ black-box VRF-combiner by contraposition. Next we inspect robust black-box combiners with additional assumptions on the candidate schemes and relaxed requirements on the construction in the hopes of finding combiner

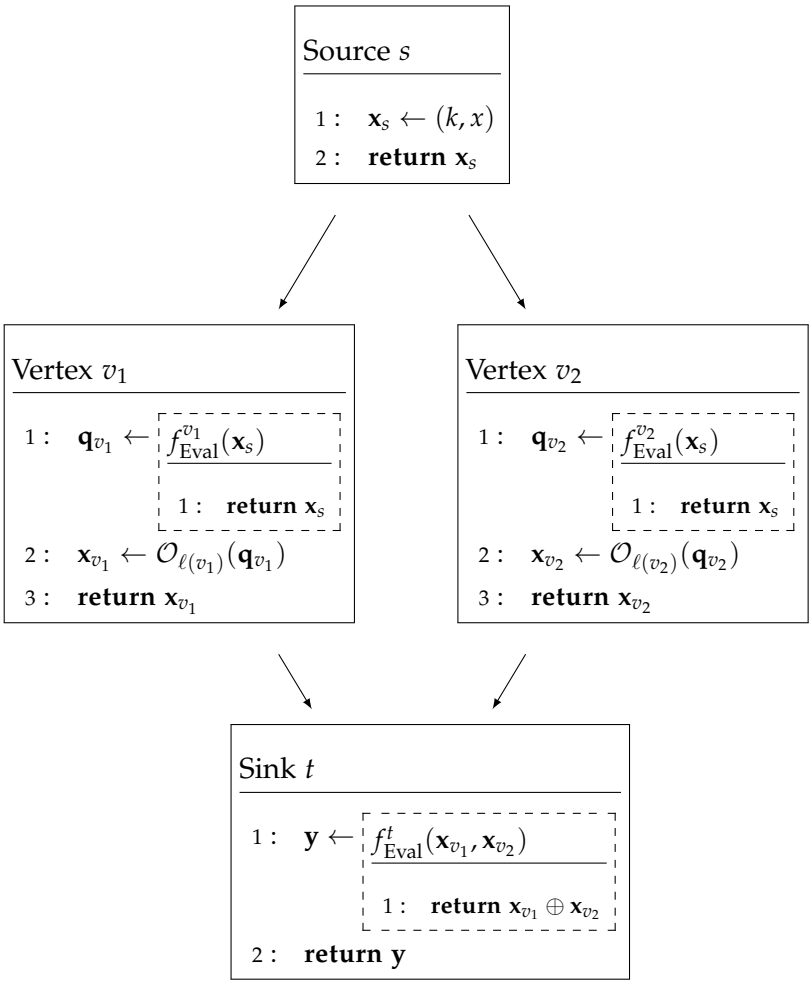


Figure 1.2: Robust $(2;1)$ Black-Box DAG PRF-combiner. Note that for every node we first combine the output of all predecessors using a ppt. Turing machine f^v_{Eval}. The output $\mathbf{w}_v$ of f^v_{Eval} is then queried to the corresponding oracle $\mathcal{O}_{\ell(v)}$. The source vertex doesn't compute anything but simply outputs the inputs while the sink t doesn't query any oracle and simply combines its inputs. In the formal definition the intermediate Turing machines f^v_{Eval} would take as input sets of tuples with a labling of the vertex and the corresponding output, e.g. $(v, \mathbf{x}_v)$, for the sake of simplicity we simply list the outputs in the arguments.

constructions. We show that even if we only require unpredictability, statistical correctness and statistical unique provability from the construction, the impossibility holds **??**, this gives two corollaries excluding the possibility of robust $(n; \lfloor n/2 \rfloor)$ black-box combiners for wVRF and VUF in Corollary 3.19 and Corollary 3.20. By strengthening the assumptions on the candidates for all candidates to implement pseudorandomness is also insufficient to find constructions **??**.

unclear

We show that if we assume all candidates to be uniquely provable and perfectly correct, the combination of the candidates works in a very similar fashion to the combiner on the PRF primitive. Note that since unique provability and perfect correctness are generally information theoretic properties, this means that for most VRF constructions, e.g. LatticeVRF [**?**] and ECVRF [**?**], those properties are typically proven mathematically. Arguable combiners are only interesting for combining computational security properties that rely on hardness assumptions.

Having strengthened the assumptions on the candidates we can now consider more advanced notions of VRFs. We'll show that with simple extensions we can construct combiners for constrainable VRFs, key-homomorphic VRFs and aggregable VRFs if all candidates are guaranteed to be uniquely provable and perfectly correct. Finally we consider unbiasable VRFs for which we provide a combiner that relies on collision resistant hash functions and deterministic 2-source randomness extractors.

Chapter 2

Notation and Preliminaries

2.1 Notation

For a positive integeger n define $[n] = \{1, \ldots, n\}$. We use $\lambda \in \mathbb{N}$ exclusively to denote the security parameter. We use $\mathsf{poly}(\lambda)$ to denote the set of functions that are polynomial in λ in the sense of Definition 2.1 and $\mathsf{negl}(\lambda)$ to denote the set of functions that are negligible in λ in the sense of Definition 2.2. We call a function f *overwhelming* if $1 - f(\lambda) \in \mathsf{negl}(\lambda)$. We use $\leftarrow$ for simple assignment. For a set S we write $s \leftarrow_\$ S$ to indicate that the value s is sampled uniformly at random from set S and for a probabilistic algorithm $\mathcal{A}$ we write $y \leftarrow_\$ \mathcal{A}(x)$ to denote assigning to y the output of $\mathcal{A}$ on input x. We use ppt. to abbreviate probabilistic polynomial time in the context of Turing machines. Finally, we use bold letters, e.g. $\mathbf{x}$, to indicate that $\mathbf{x}$ consists of multiple values such as tuples or vectors.

maybe change

Definition 2.1 (Polynomial Functions) *We say that a function $p : \mathbb{N} \to \mathbb{R}$ is polynomial if the folowing holds*

$$\exists d \in \mathbb{N}, \exists n_0 \in \mathbb{N}, \forall n > n_0, p(n) \leq n^d$$

Definition 2.2 (Negligible Functions) *We say that a function $\varepsilon : \mathbb{N} \to \mathbb{R}$ is negligible if the following holds*

$$\forall p \in \mathsf{poly}(\lambda), \exists n_0 \in \mathbb{N}, \forall n > n_0, |\varepsilon(n)| < \frac{1}{p(n)}$$

2.2 Verifiable Functions

A verifiable function (VF) VF is a cryptographic primitive that implements algorithms $\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}$ and $\mathsf{Vfy}_{\mathsf{VF}}$. A user can generate key-pairs $(\mathsf{sk}, \mathsf{pk})$ using $\mathsf{Gen}_{\mathsf{VF}}$ consisting of a secret key sk and a public key pk, use the secret key sk to make evaluate arbitrary points $x \in \{0,1\}^\lambda$ using $\mathsf{Eval}_{\mathsf{VRF}}$ and use the public key pk to verify a triple (x, y, π) using $\mathsf{Vfy}_{\mathsf{VRF}}$. We give a formal definition in Definition 2.3, the definition takes inspiration of the definition of a VRF in [**?**]. To reduce redundancy we define additional security properties such as pseudorandomness and unpredictability seperately.

Definition 2.3 (Verifiable Function) *A verifiable function (VF)* VF *is a triple of efficient (ppt.) algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *with the syntax*

this is less elegant now

- $\mathsf{Gen}_{\mathsf{VF}}$*: The key-generation algorithm takes as input the unary encoding of the security parameter and produces a pair* $(\mathsf{sk}, \mathsf{pk}) \in \mathcal{SK} \times \mathcal{PK}$ *containing a secret and a public key respectivley.*
- $\mathsf{Eval}_{\mathsf{VF}}$*: The evaluation algorithm takes as input a tuple* (sk, x) *containing a secret key* $\mathsf{sk} \in \mathcal{SK}$ *and an evaluation point* $x \in \{0,1\}^\lambda$ *and produces an tuple* (y, π) *containing an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$*.*
- $\mathsf{Vfy}_{\mathsf{VF}}$*: The verification algorithm takes as input a tuple* (pk, x, y, π) *containing a public key* $\mathsf{pk} \in \mathcal{PK}$*, an evaluation point* $x \in \{0,1\}^\lambda$*, an image* $y \in \mathcal{Y}$ *and a proof* $\pi \in \mathcal{P}$ *and returns a bit* b *denoting acceptance or rejection.*

can evaluation or verification be randomized, probably vfy can be probabilistic

We use the notation $\mathsf{Eval}^y_{\mathsf{VF}}$ and $\mathsf{Eval}^\pi_{\mathsf{VF}}$ to denote the algorithm $\mathsf{Eval}_{\mathsf{VF}}$ to projected on the respective output. Note that a VF is a purely syntactical concept. For a VF we define various properties to arrive to arrive at specific cryptographic primitives, e.g. VRFs. First of all we define ε-correctness, a property that guarantes that over random key-pairs and evaluation points, the verification algorithm accepts a correctly computed image-proof pair with probability at least ε.

Definition 2.4 (ε-Correctness, [?]) *A verifiable function* $\mathsf{VF} = (\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is* ε*-correct, if for all security parameters* $\lambda \in \mathbb{N}$ *we have that*

$$\Pr\left[\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = 1 : \begin{array}{c}(\mathsf{sk}, \mathsf{pk}) \leftarrow_\$ \mathsf{Gen}_{\mathsf{VF}}(1^\lambda) \\ x \leftarrow_\$ \{0,1\}^\lambda \\ (y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)\end{array}\right] \geq \varepsilon(\lambda)$$

We define a VF VF *to be perfectly correct, or just correct, if it is* 1*-correct and statistically correct if* ε *is an overwhelming function.*

Intuitively a correct VF states that honestly generated image-proof pairs are accepted by the verification algorithm. Next we define unique provability. A

VF is uniquely provable if not more than one image can be proven, even for maliciously crafted public keys.

Definition 2.5 (Unique Provability, [?]) *A verifiable function* VF*, with algorithms* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$*, is defined to be uniquely provable if for all evaluation points* $x \in \{0,1\}^\lambda$*, all public-keys* $\mathsf{pk} \in \mathcal{PK}$ *and any two images-proof pairs* $(y_0, \pi_0), (y_1, \pi_1) \in \mathcal{Y} \times \mathcal{PK}$ *we have:*

$$\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_0, \pi_0) = 1 \land \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1$$

maybe comment why no statistical notion

Correctness and unique provability are information theoretic. We define pseudorandomness of a VF which is a computational property, stating that no efficient adversary $\mathcal{A}$ can distinguish images y corresponding to evaluation of x, chosen by $\mathcal{A}$, from a uniformly random output $y^* \leftarrow_\$ \mathcal{Y}$. This needs to hold even if the adversary gets to see the evaluation of a polynoial number of points $x \neq x^*$.

Definition 2.6 ((t, ε)-Pseudorandomness, [?]) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t, ε)*-pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the prf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is pseudorandom, if it is (t, ε)*-pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Next we define two weaker notions of pseudorandomness. While (strong) pseudorandomness required the evaluation of adverserially chosen points to be indistinguishable from uniformly random elements $y^* \in \mathcal{Y}$ we can relax this requirement by requiring computational indistinguishability for uniformly random challenges $x^* \leftarrow_\$ \{0,1\}^\lambda$. We call this notion weak pseudorandomness.

Definition 2.7 ((t, ε)-Weak Pseudorandomness, [?]) *A VF* $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ *is defined to be* (t, ε)*-weakly pseudorandom if for all security parameters* $\lambda \in \mathbb{N}$*, for all stateful adversaries* $\mathcal{A}$*, running in time at most* $t(\lambda)$*, the advantage of* $\mathcal{A}$ *in the wprf security experiment outlined in Figure 2.1 is at most* $\varepsilon(\lambda)$*, i.e. if*

$$\mathbf{Adv}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is weakly pseudorandom, if it is (t, ε)*-weakly pseudorandom with* ε *being a negligible function and* t *being a polynomial, i.e.* $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$*.*

Alternatively a VF can be unpredictable. A VF is unpredictable if no adversary can predict the image y of the evaluation of a challenge x chosen by the adversary. This property needs to hold even if the adversary sees the evaluation of a polynomial number of points $x \neq x^*$. Note that if the adversary can predict the output of evaluation on x^* he can also distinguish the output from a random output $y^* \leftarrow_{\$} \{0,1\}$ with non-negligible probability making unpredictability is a weaker property than pseudorandomness.

Definition 2.8 ((ε, t)-Unpredictability, [?]) *A VF $(\mathsf{Gen}_{\mathsf{VF}}, \mathsf{Eval}_{\mathsf{VF}}, \mathsf{Vfy}_{\mathsf{VF}})$ is defined to be (t, ε)-unpredictable if for all security parameters $\lambda \in \mathbb{N}$, for all stateful adversaries $\mathcal{A}$, running in time at most $t(\lambda)$, the advantage of $\mathcal{A}$ in the up security experiment outlined in Figure 2.1 is at most $\varepsilon(\lambda)$, i.e. if*

$$\mathbf{Adv}^{\mathrm{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

We say that a VF is unpredictable, if it is (t, ε)-unpredictable with ε being a negligible function and t being a polynomial, i.e. $\varepsilon \in \mathsf{negl}(\lambda), t \in \mathsf{poly}(\lambda)$.

We now define the notions of a verifiable random function (VRF), verifiable unpredictable function (VUF) and weak verifiable random function (WVRF), which are correct, uniquely provable and pseudorandom, unpredictable and weakly pseudorandom VFs respectivly. We formally define the primitives.

Definition 2.9 (Verifiable Random Functions) *A verifiable random function $\mathsf{VRF} = (\mathsf{Gen}_{\mathsf{VRF}}, \mathsf{Eval}_{\mathsf{VRF}}, \mathsf{Vfy}_{\mathsf{VRF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and pseudorandomness in the sense of Definition 2.6.*

Definition 2.10 (Weak Verifiable Random Functions) *A weak verifiable unpredictable function $\mathsf{WVRF} = (\mathsf{Gen}_{\mathsf{WVRF}}, \mathsf{Eval}_{\mathsf{WVRF}}, \mathsf{Vfy}_{\mathsf{WVRF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and weak pseudorandomness in the sense of Definition 2.7.*

Definition 2.11 (Verifiable Unpredictable Functions) *A verifiable unpredictable function $\mathsf{VUF} = (\mathsf{Gen}_{\mathsf{VUF}}, \mathsf{Eval}_{\mathsf{VUF}}, \mathsf{Vfy}_{\mathsf{VUF}})$ is a VF that satisfies perfect correctness in the sense of Definition 2.4, unique provability in the sense of Definition 2.5 and unpredictability in the sense of Definition 2.8.*

Experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $\sigma \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $x^* \leftarrow_{\$} \{0,1\}^{\lambda}$
6 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
7 : $y_1 \leftarrow_{\$} \mathcal{Y}$
8 : $b' \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, x^*, \sigma, y_b)$
9 : **return** $b = b' \wedge x^* \notin S$

Experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
2 : $S \leftarrow \varnothing$
3 : $(x^*, y^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{eval}}}(1^{\lambda}, \mathsf{pk})$
4 : $(y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
5 : **return** $y^* = y \wedge x^* \notin S$

Oracle $\mathcal{O}_{\text{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Figure 2.1: The pseudorandomness security experiment $\mathbf{Exp}^{\text{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.6, the weak pseudorandomness security experiment $\mathbf{Exp}^{\text{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.7 and the unpredictability security experiment $\mathbf{Exp}^{\text{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$ used in Definition 2.8. The main changes between the pseudorandomness security experiment and the weak pseudorandomness security experiment are highlighted in grey.

2.3 Robust Combiners

For two natural numbers n, k we define the robust $(n;k)$ black-box combiner, or robust black-box combiner for short. A robust $(n;k)$ black-box combiner for a primitive P is a construction Π_{P}, to be precise a ppt. oracle Turing machine, that is given query access to n oracles, the candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$. If at least k candidate schemes implement the primitive P, then the combiner $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}$ securely implements P. The idea behind a robust black-box combiner is that the construction extracts the security properties from the subset of correct candidates, even if some candidate schemes are not secure instances. However now note the following problem in the current definition:

If there exists a secure instance of P there always exists robust $(n;0)$ black-box combiner that simply ignores its candidates and implements P. This robust black-box combiner meets all the requirements of the definition. Showing

that there exists no specific robust black-box combiner therefore requires to show that no secure instance of the primitive P exists, which is often a difficult complexity theoretic statement.

To circumvent this problem we want to explicitly force the combiner to derive its security from the subset of secure candidate schemes. To do this, we require the security properties of the combiner to hold relative to a PSPACE complete oracle. Firstly we want to note that this means that the adversary against the combiner can efficiently check whether an element is a member of a PSPACE complete language, enabling the adversary to generally break computational hardness assumptions but limiting the adversary still to a polynomial number of oracle queries. Secondly we want to note that this notion of a combiner still doesnt solve the issue if the security of the cryptographic primitive are *all* information theoretic.

Very unhappy with this definition

TODO state contrapositive of the definition below here.

Definition 2.12 (Robust $(n;k)$ Black-Box P-Combiner) *A robust $(n;k)$ black-box combiner for primitive* P *with k algorithms is a tuple of ppt. oracle Turing machines $\Pi_{\mathsf{P}} = (\mathsf{Alg}_{\mathsf{P}}^1, \ldots, \mathsf{Alg}_{\mathsf{P}}^k)$ such that if there exists an ppt. adversary $\mathcal{A}^{\mathcal{O}_1,\ldots,\mathcal{O}_n,\mathsf{PSPACE}}$ that against $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ then there exist $n-k+1$ ppt. adversaries $\mathcal{B}_1^{\mathcal{O}_{\ell(1)}}, \ldots, \mathcal{B}_{n-k+1}^{\mathcal{O}_{\ell(n-k+1)}}$ and an injective function $\ell : [n-k+1] \to [n]$ such that for any $i \in [n-k+1]$ the adversary $\mathcal{B}_i^{\mathcal{O}_{\ell(i)}}$ breaks the security of the candidate $\mathcal{O}_{\ell(i)}$.*

Remark. If P defines composite security properties, as is the case for VRFs for instance, we say that an adversary breaks the security of P if it breaks one of the required security properties. For computational security properties such as pseudorandomness this typically means that the adversary wins the corresponding security experiment with non-negligible advantage. For information theoretic or statistical properties such as unique provability or correctness this means that the property doesn't hold (we can think of this as the adversary $\mathcal{A}$ being arbitrary, breaking the security).

Chapter 3

Results

3.1 Robust Black-Box DAG Combiner

Intuitively, a robust black-box combiner operates on n candidate schemes by adaptively interleaving calls to the candidates oracles with ppt. processing to combine the candidates outputs in a way such that security properties of the secure subset of oracles are preserved if there are sufficiently many of them. For certain arguments it can be helpful to model the combiners structure more explicitly, we do so by modeling every algorithm implemented by the combiner with a directed acyclic graph (DAG) reflecting the computation.

In addition to the DAG we specify dedicate source and sink vertices s and t, a vertex labling $\ell : V(\mathcal{G}) \setminus \{s,t\} \to [n] \times [|\mathsf{P}|]$ mapping every non-source and non-sink vertex to a tuple (i,j) indicating which candidate is queried on which oracle and ppt. Turing machines f^v for every vertex $v \in V(\mathcal{G})$ that compute the query input to the candidate and oracle $\ell(v)$ given the oracle response of the predecessors $\mathcal{N}_{\mathcal{G}}^{-}(v)$. The source vertex s simply corresponds to the input $\mathbf{x}$ to the algorithm while the target is associated with a single ppt. Turing machine f^t that computes the final output of the combiner $\mathbf{y}$.

A robust black-box DAG combiner algorithm, is then executed by tracing the computation through the corresponding DAG $\mathcal{G}$. The input to every oracle query is computed from the response of the oracles at the preceding vertices using the Turing machine f^v associated with the respective vertex $v \in V(\mathcal{G})$. The output of the algorithm is given by the output of f^t on the respective inputs. We assume that the random tape of the intermediate Turing machines are initialized identically to the random tape of the combiner Turing machine . A robust black-box DAG combiner has to satisfy that the algorithm induced by the DAG description needs to be a robust black-box combiner. We formally define robust black-box DAG combiners as follows, we use $|\mathsf{P}|$ for primitive P to denote the number of implemented algorithms and generically name them $(\mathsf{Alg}_{\mathsf{P}}^1, \ldots, \mathsf{Alg}_{\mathsf{P}}^p)$ if $|\mathsf{P}| = p$.

argue

Definition 3.1 (Robust $(n;k)$ Black-Box DAG P-Combiner) *A roubst $(n;k)$ black-box DAG combiner for primitive P is a robust $(n;k)$ black-box algorithm Π_{P} in the sense of Definition 2.12, i.e. a tuple algorithms $(\mathsf{Alg}^1_{\mathsf{P}}, \ldots, \mathsf{Alg}^p_{\mathsf{P}})$ where every algorithm $\mathsf{Alg}^i_{\mathsf{P}}$ is specified by a tuple of the following shape*

$$\left(\mathcal{G}_p, s_p, t_p, \ell_p, \{f_p^v\}_{v \in V(\mathcal{G}_p) \setminus \{s_p\}}\right)$$

Such that for all $p \in \mathsf{P}$, $\mathcal{G}_p$ is a DAG, s_p and t_p are vertices in $V(\mathcal{G}_p)$ such that s_p has no ingoing edges and t_p has no outgoing edges, $\ell_p : V(\mathcal{G}_p) \to [n] \times \mathsf{P}$ is a vertex labling and $\{f_p^v\}_{v \in V(\mathcal{G}_p) \setminus \{s_p\}}$ are pt. Turing machines. Where, given the tuple, the algorithm $\mathsf{Alg}^p_{\mathsf{P}}$ is defined as shown in Figure 3.1.

Algorithm $\mathsf{Alg}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}(\mathbf{x})$

$\mathbf{x}_s \leftarrow \mathbf{x}$
for $v \in \mathsf{topo}(V(\mathcal{G}_p) \setminus \{s_p, t_p\})$ **do**
  // Compute the query $\mathbf{q}_v$ to the oracle
  $\mathbf{q}_v \leftarrow f_p^v(\{(w, \mathbf{x}_w)\}_{w \in \mathcal{N}^-_{\mathcal{G}_p}(v)})$
  // Evaluate the oracle on $\mathbf{q}_v$
  $\mathbf{x}_v \leftarrow \mathcal{O}_{\ell(v)}(\mathbf{q}_v)$
return $f_p^t(\{w, \mathbf{x}_w\}_{w \in \mathcal{N}^-_{\mathcal{G}_p}(t)})$

Figure 3.1: Definition of the robust black-box DAG combiner algorithm $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{P}}.\mathsf{Alg}_{\mathsf{P}}$ initialized with candidates $\mathcal{O}_1, \ldots, \mathcal{O}_n$. We use topo to denote an algorithm returning the vertices in $V(\mathcal{G}_p)$ in a fixed topological ordering. We use the notation $\mathcal{O}_{c(v)}$ for $c(v) = (i, j)$ to denote the oracle j or the i'th candidate $\mathcal{O}_i$, i.e. $\mathcal{O}_i.\mathsf{Alg}_{\mathsf{P},\mathsf{j}}$.

By definition, a robust black-box DAG combiner corresponds to a robust black-box combiner. We also show that for every robust black-box combiner there exists a canonical robust black-box DAG combiner which is *computationally equivalent* in the sense that the algorithms implemented realize equivalent functions. We state this result in the following theorem, which we will be using to lead our impossibility proofs over robust black-box DAG combiners instead of robust black-box combiners.

computational equivalence more def?

Theorem 3.2 (Robust $(n;k)$ Black-Box DAG Combiners) *For every robust $(n;k)$ black-box combiner in the sense of Definition 2.12 there exists a computationally equivalent robust $(n;k)$ black-box DAG combiner in the sense of Definition 3.1.*

Intuition. The idea behind the proof is to take an arbitrary robust $(n;k)$ black-box combiner Π_{P} and unroll its computation in a graph consisting of $T(n,\lambda)$ layers where each vertex on a layer k is connected to all vertices of the preceding layers $i < k$. Each layer k consisting of $n|\mathsf{P}|$ vertices corresponding

to the different oracles which can be queried as the k'th query. At every layer k for every vertex the combiner then simulates the Turing machine of Π_{P} until the $k+1$'the oracle query. Since the Turing machine f^v can however not make any oracle query itself it relies on the previous connected vertices to obtain the answer to those oracle queries. For the i'th query the Turing machine simply uses the output of the correct vertex on layer i to simulate the oracle query. Once we reach the k'th query and the query would be to oracle $\mathcal{O}_{(i,j)}$ then we distinguish the case when we are at the correct vertex or not. If the current vertex satisfies $c(v) = (i,j)$ then we simply return the query $\mathbf{q}_v$ resulting in the correct query being made, otherwise if $c(v) \neq (i,j)$ then we simply return an arbitrary query that meets the expected input format of $\mathcal{O}_{c(v)}$. An important nuance is that we need to fix the random tape of the Turing machines ρ since otherwise between simulating the oracle queries at a later point the executions may diverge.

maybe cover more explicitly

Proof (Theorem 3.2) Let Π_{P} be a fixed robust $(n;k)$ black-box combiner for primitive P and let $\mathsf{Alg}_{\mathsf{P}}$ be an arbitrary algorithm implemented by P. We label the candidate schemes with $\mathcal{O}_1, \ldots, \mathcal{O}_n$. We construct a robust $(n;k)$ black-box DAG combiner $\widetilde{\Pi}_{\mathsf{P}}$ that is computationally equivalent to Π_{P} to prove the theorem.

Since Π_{P} is a robust $(n;k)$ black-box combiner it is a ppt. Turing machine and the number of oracle queries made by Π_{P} can therefore be bounded from above by $Q(\lambda, n)$, a polynomial in λ and n. We construct the DAG $\mathcal{G}_{\mathsf{Alg}_{\mathsf{P}}} = (V_{\mathsf{Alg}_{\mathsf{P}}}, E_{\mathsf{Alg}_{\mathsf{P}}})$ as follows:

$$V_{\mathsf{Alg}_{\mathsf{P}}} = \{s_{\mathsf{Alg}_{\mathsf{P}}}, t_{\mathsf{Alg}_{\mathsf{P}}}\} \cup [Q(\lambda,n)] \times [n] \times [|\mathsf{P}|] \tag{3.1}$$

$$E_{\mathsf{Alg}_{\mathsf{P}}} = \{((d_1,i_1,j_1),(d_2,i_2,j_2)) \in [Q(\lambda,n)] \times [n] \times [|\mathsf{P}|] : d_1 < d_2\} \tag{3.2}$$

$$\cup \{s_{\mathsf{Alg}_{\mathsf{P}}}\} \times [Q(\lambda,n)] \times [n] \times [|\mathsf{P}|] \tag{3.3}$$

$$\cup [Q(\lambda,n)] \times [n] \times [|\mathsf{P}|] \times \{t_{\mathsf{Alg}_{\mathsf{P}}}\} \tag{3.4}$$

We define a vertex labling $\ell_{\mathsf{Alg}_{\mathsf{P}}}(d,i,j) = (i,j)$ and the ppt. Turing machines $\{f^v_{\mathsf{Alg}_{\mathsf{P}}}\}_{v \in V_{\mathsf{Alg}_{\mathsf{P}}}}$ as follows: On input $\{(w, \mathbf{x}_w)\}_{w \in \mathcal{N}_{\mathcal{G}}^-}$ simulate the Turing machine $\Pi_{\mathsf{P}}.\mathsf{Alg}_{\mathsf{P}}$ on input $\mathbf{x}$. For every q'th query made to the oracle (i,j) it looks up the value of $\mathbf{x}_{(q,i,j)}$ and continues its execution with that oracle response. For the k'th query if the query is to oracle $c(v)$ then simply return the result, otherwise if the query is to an oracle $(i,j) \neq c(v)$ then return an element in the domain of the oracle (i,j) (that oracle query will not further be considered in the execution). For $f^t_{\mathsf{Alg}_{\mathsf{P}}}$ simulate the entire execution of $\Pi_{\mathsf{P}}.\mathsf{Alg}_{\mathsf{P}}$ and proceed analogously but since no more query will be made we just return the result. To show that $\widetilde{\Pi}_{\mathsf{P}}$ is a robust $(n;k)$ black-box DAG combiner we need to show that the algorithm $\mathsf{Alg}_{\mathsf{P}}$ specified by $(\mathcal{G}_{\mathsf{Alg}_{\mathsf{P}}}, s_{\mathsf{Alg}_{\mathsf{P}}}, t_{\mathsf{Alg}_{\mathsf{P}}}, \ell_{\mathsf{Alg}_{\mathsf{P}}}, \{f^v_{\mathsf{Alg}_{\mathsf{P}}}\}_{v \in V(\mathcal{G}_{\mathsf{Alg}_{\mathsf{P}}})})$ is computationally equivalent to $\Pi_{\mathsf{P}}.\mathsf{Alg}_{\mathsf{P}}$ and runs in time polynomial in λ, n. We first show that the algorithm is computationally equivalent, for that consider the following claim.

how to best define

Claim 3.3 *Let Π_{P} be a robust $(n;k)$ black-box combiner for primitive in the sense of Definition 2.12 and let $\widetilde{\Pi}_{\mathsf{P}}$ be the algorithm obtained by applying the procedure above, then we have for for all candidates $\mathcal{O}_1,\ldots,\mathcal{O}_n$ and every algorithm $\mathsf{Alg}^i_{\mathsf{P}}$ implemented by P the following holds.*

$$\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{Alg}^i_{\mathsf{P}} = \widetilde{\Pi}_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{Alg}^i_{\mathsf{P}}$$

Proof (Claim 3.3) Fix a robust $(n;k)$ black-box P-combiner Π_{P}, candidate schemes $\mathcal{O}_1,\ldots,\mathcal{O}_n$, an algorithm implemented by Π_{P}, let $\widetilde{\Pi}_{\mathsf{P}}$ be the robust $(n;k)$ black-box DAG P-combiner constructed as specified above and let $\mathbf{x}$ be an arbitrary input to the algorithm $\mathsf{Alg}_{\mathsf{P}}$. We show that

$$\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{Alg}^i_{\mathsf{P}}(\mathbf{x}) = \widetilde{\Pi}_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{Alg}^i_{\mathsf{P}}(\mathbf{x})$$

By induction over the number of queries made by the Π_{P} combiner. We say that an vertex v on layer $d \in [Q(\lambda,n)]$ is *correct* if in the execution of $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}.\mathsf{Alg}^i_{\mathsf{P}}(\mathbf{x})$ the d'th oracle query is made to the oracle $c(v)$ and we say that its *incorrect* otherwise. Formally we show that for every $d \in [Q(\lambda,n)]$ the output of the d'th query of the robust black-box combiner $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ and the output of the d'th query of the robust black-box DAG combiner $\widetilde{\Pi}_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ agree.

Base case: If $d = 1$ then the output of the robust black-box DAG combiner $\widetilde{\Pi}_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ is computed by simulating the evaluation of $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ until the first oracle query and then making the correct oracle query itself at the correct vertex. Therefore the base case holds.

Inductive step: We assume that for every $d' < d$ the output of the query at the correct vertex v at layer d' is equivalent to the output of the d''th oracle query output are the same. Since therefore the execution of the algorithm of $\widetilde{\Pi}_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ is perfectly simulated it computes the exact same input and queries by correctness of the vertex the correct oracle on the same input, resulting in the same output. □

what about randomized oracles

Claim 3.4 *Let Π_{P} be a robust $(n;k)$ black-box combiner for primitive in the sense of Definition 2.12 and let $\widetilde{\Pi}_{\mathsf{P}}$ be the algorithm obtained by applying the procedure above, then we have for for all candidates $\mathcal{O}_1,\ldots,\mathcal{O}_n$ and every algorithm $\mathsf{Alg}^i_{\mathsf{P}}$ runs in polynomial time.*

Proof (Claim 3.4) By construction the DAG $\mathcal{G}_{\mathsf{Alg}^i_{\mathsf{P}}}$ has $2 + n|\mathsf{P}|Q(\lambda,n)$ vertices. At each vertex $v \in V(\mathcal{G}^i_{\mathsf{Alg}_{\mathsf{P}}})$ the Turing machine $f^v_{\mathsf{Alg}^i_{\mathsf{P}}}$ simulates $\Pi_{\mathsf{P}}.\mathsf{Alg}_{\mathsf{P}}$ where every oracle query is answered with a constant-time lookup along predecessor outputs, i.e. runs in time a polynomial $T(n,\lambda)$. Hence the overall running time is bounded by

$$|V(\mathcal{G}_{\mathsf{Alg}_{\mathsf{P}}})|T(\lambda,n) \in \mathsf{poly}(\lambda,n)$$

Concluding the proof. □

It remains to show that $\widetilde{\Pi}_{\mathsf{P}}$ is indeed a robust $(n;k)$ black-box combiner. By Claim 3.3 the algorithms implemented by $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ and $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ are equivalent. In particular, any adversary $\mathcal{A}^{\mathcal{O}_1,\ldots,\mathcal{O}_n,\mathsf{PSPACE}}$ against $\widetilde{\Pi}_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ is also an adversary against $\Pi_{\mathsf{P}}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}$ with the same advantage. By Definition 2.12 this means that there exist $n-k+1$ ppt. adversaries $\mathcal{B}_1^{\mathcal{O}_{\ell(1)}},\ldots,\mathcal{B}_{n-k+1}^{\mathcal{O}_{\ell(n-k+1)}}$ and an injective function $\ell : [n-k+1] \to [n]$ such that for any $i \in [n-k+1]$ the adversary $\mathcal{B}_i^{\mathcal{O}_{\ell(i)}}$ breaks the security of the candidate $\mathcal{O}_{\ell(i)}$. Hence $\widetilde{\Pi}_{\mathsf{P}}$ is a robust $(n;k)$ black-box combiner. □

Example. For an example consider of a robust $(2;1)$ black-box DAG combiner for the PRF primitive see Figure 1.2. Note that the construction shown in Figure 1.2 doesn't correspond to the robust $(2;1)$ black-box DAG PRF-combiner one would obtain by following the procedure used in the proof of Theorem 3.2.

3.2 Impossibility of Robust Black-Box VRF-Combiner

We consider the possibility of combining VRFs in a black-box fashion. Specifically, we study the possibilities of combining two candidates, at least one of them implementing a secure VRF. Such robust $(2;1)$ black-box combiners are the simplest robust black-box combiner construction. However the existence of robust $(2;1)$ black-box combiners immediately give rise to the robust $(n;1)$ black-box combiners, while an impossibility implies that there exists no robuts $(n;\lfloor n/2 \rfloor)$ black-box combiner. Consequently, settling the $(2;1)$ case in either direction extends to a seemingly stronger result.

In this section we present a proof of the impossibility of robust $(2;1)$ black-box VRF-combiners. At a high level, the proof exploits the fact that the oracles outputs must be used to produce a pseudorandom output but at the same time cannot influence the output distribution "too much" since otherwise if the candidate were the malleable candidate he could allow for two distinct images to be produced without the verifier being able to tell the difference. In the proof we make the simplifying assumption that the secret key space of all candidates is given by $\mathcal{SK}_i = \{0,1\}^{s_i(\lambda)}$ for a function $s_i \in \mathsf{poly}(\lambda)$. We state the formal statement in the following theorem.

Theorem 3.5 (Robust (2;1) Black-Box VRF Combiner) *There exists no robust $(2;1)$ black-box combiner as defined in Definition 2.12 for the VRF primitive as defined in Definition 2.9.*

Intuition. Instead of showing the impossibility for the robust $(2;1)$ black-box combiner, we show an impossibility for robust $(2;1)$ black-box DAG combiners. The impossibility of a robust $(2;1)$ black-box VRF-combiner then follows directly from the equivalence of robust black-box combiners and robust black-box DAG combiners in Theorem 3.2.

Now suppose, for a contradiction, that there exists a robust $(2;1)$ black-box DAG VRF-combiner Π_{VRF}. Let $\mathcal{O}_1, \mathcal{O}_2$ be candidate schemes, at least one of which implementing a secure VRF. By Definition 2.12, $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ must implement a secure VRF. In particular, since $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ implements a secure VRF, it must be perfectly correct and uniquely provable. We derive a contradiction by initializing $\mathcal{O}_1, \mathcal{O}_2$ with specific candidate schemes of which at least one implements a secure VRF, and then constructing an adversary $\mathcal{A}^{\mathsf{PSPACE},\mathcal{O}_1,\mathcal{O}_2}$ that breaks the pseudorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ with non-negligible advantage.

The adversary works by iteratively narrowing down the set of candidate secret keys in the pseudorandomness experiment, starting with the secret key space $\mathcal{SK}$, by relying on its PSPACE complete oracle PSPACE. The filtering works as follows: $\mathcal{A}$ samples a polynomial number of evaluation points $x_1, \ldots, x_{2s(\lambda)}$ u.a.r. from $\{0,1\}^\lambda$. Then uses the evaluation oracle $\mathcal{O}_{eval}$ in the prf security

experiment to evaluate the points under the games secret key sk^* to obtain image-proof pairs $(y_1, \pi_1), \ldots, (y_{2s(\lambda)}, \pi_{2s(\lambda)})$. Using the games public key pk^* the adversary now computes the trace of the combiners verification algorithm on the tuples $(\mathsf{pk}^*, x_1, y_1, \pi_1), \ldots, (\mathsf{pk}^*, x_{2s(\lambda)}, y_{2s(\lambda)}, \pi_{2s(\lambda)})$ to obtain $\tau_1, \ldots, \tau_{2s(\lambda)}$. The key ingredient is a ppt. Turing machine $\mathsf{EvalSim}_{\mathsf{VRF}}$ that takes as input a tuple $(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i)$ and returns either $\top$ or $\bot$, satisfying the following properties:

- $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes no oracle queries to the candidates $\mathcal{O}_1, \mathcal{O}_2$
- If $\mathsf{sk} = \mathsf{sk}^*$ then $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top$
- If $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x_i) \neq y_i$ then $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \bot$

Before elaborating on the construction of $\mathsf{EvalSim}_{\mathsf{VRF}}$ we first explain how the adversary can use this ppt. Turing machine to break pseudorandomness of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$. Recall that we require the security properties of the combiner to hold relative to a PSPACE complete oracle precisely because we want the combiner to derive its security from the candidate oracles (and not implement it itself). Now, $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes no oracle queries yet still only returns $\top$ if a secret key candidate $\mathsf{sk} \in \mathcal{SK}$ *agrees* with the games secret key sk^* in the sense that on x_i they evaluate to the same image. Using its PSPACE complete oracle PSPACE the adversary can now enumerate the entire secret key space $\mathcal{SK}$ and discard any secret key candidate for which there exists a tuple $(x_i, y_i, \pi_i, \tau_i)$ such that $\mathsf{EvalSim}_{\mathsf{VRF}}$ returns $\bot$ (strictly speaking, the PSPACE oracle decides membership queries for PSPACE languages, we defer these technicalities to the formal proof). Note that sk^* is never eliminated since $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}^*, x_i, y_i, \pi_i, \tau_i) = \top$ always and that all other survivors satisfy that they evaluate to the same image as sk^* under $x_1, \ldots, x_{2s(\lambda)}$. There are two possible cases that can happen after the narrowing has taken place:

If for sufficiently many tuples $(x_i, y_i, \pi_i, \tau_i)$ the number of eliminated candidate secret keys is sufficiently large then after $2s(\lambda)$ iterations of filtering we end up with a polynomially sized set $\mathcal{SK}_{cand}$ containing sk^*. The adversary can then sample a uniformly random secret key $\mathsf{sk}_{cand} \in \mathcal{SK}_{cand}$, winning the security experiment if $\mathsf{sk}^* = \mathsf{sk}_{cand}$. This happens with non-negligible probability.

On the other hand if only for a small number of tuples $(x_i, y_i, \pi_i, \tau_i)$ the size of the set of candidate keys decreases significantly this means that for many of the remaining secret key candidates, the probability of evaluating to the same image than the experiments secret key on uniformly random evaluation points is non-negligible. In this scenario the adversary proceeds analogously, he samples a uniformly random secret key candidate and evaluates the challenge pre-image x^* to compare with the challenge image y^*. Since sk_{cand} agrees with the expeirments secret key sk^* with non-negligible probability the adversary wins the game with non-negligible advantage.

We now describe the construction of $\mathsf{EvalSim}_{\mathsf{VRF}}$ algorithm. The algorithm simulates the evaluation of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ by iterating over the the vertices in $V(\mathcal{G}_{\mathsf{Eval}_{\mathsf{VRF}}})$ in a fixed but arbitrary topological ordering. At every vertex v it simulates the associated Turing machine f_{eval}^{v} on the corresponding inputs to obtain $\mathbf{q}_v$. Oracle queries to $\mathcal{O}_1$ are answered trivially with a fixed response, this is the case as we will later initialize this oracle with a trivial, uniquely provable and perfectly correct oracle that returns fixed values for key generation and evaluation and verifies exactly the fixed image. For oracle queries to $\mathcal{O}_2$ we distinguish *exposed* queries, which are contained in the verification trace τ_i, and *obscure* queries, which are not contained in the verification trace. The simulator answers the exposed queries with the response contained in τ_i. Thus the only queries that cannot perfectly be answered are the obscure queries to $\mathcal{O}_2$.

For every obscure query to $\mathcal{O}_2$ the adversary branches its execution for every well-formed responses of the query (e.g. all possible values in $\mathcal{Y} \times \mathcal{P}$ if the query is to $\mathcal{O}_2$s evaluation algorithm) and continues the execution for every branch. Finally if for all branches the evaluation of the combiner is y_i $\mathsf{EvalSim}_{\mathsf{VRF}}$ returns $\top$, otherwise it returns $\bot$. We illustrate the algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}$ in Figure 3.2.

Its easy to see that the first and third requirements for the $\mathsf{EvalSim}_{\mathsf{VRF}}$ algorithm are met. By construction $\mathsf{EvalSim}_{\mathsf{VRF}}$ makes no oracle queries to candidates. Assuming that $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x_i) \neq y_i$ then, since we answer all queries to $\mathcal{O}_1$ and the exposed queries to $\mathcal{O}_2$ truthfully, there must exits a well-formed assignment of oracle responses to obscure queries to candidate $\mathcal{O}_2$ such that the execution of $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x_i)$ is simulated perfectly and therefore the output is not y_i, thus we have $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \bot$.

For the second property, we argue that for the security experiments secret key sk^* any assignment of responses to obscure queries to candidate $\mathcal{O}_2$ does not change the image produced by the combiner simulation. Suppose for a contradiction that there exists two different assignments of oracle responses that result in two different images $\tilde{y} \neq \hat{y}$. We derive a contradiction in three steps. First, we may assume without loss of generality that in the evaluation of the robust black-box DAG combiners evaluation algorithm every specific oracle is queried on a specific input at most once (Claim 3.6). Secondly, we construct two oracles $\widehat{\mathcal{O}}_2, \widetilde{\mathcal{O}}_2$, possibly not uniquely provable or perfectly correct, which are the oracle $\mathcal{O}_2$ remapped on the obscure queries to the two assignments that produce different images $\hat{y} \neq \tilde{y}$ respectively. Third, we turn $\mathcal{O}_1$ into a full VRF by remapping all but a small subset of evaluation points to agree with the secure VRF $\mathcal{O}_2$ (Lemma 3.7). Since $\widehat{\mathcal{O}}_2, \widetilde{\mathcal{O}}_2$ produce different images but only differ on obscure queries not contained in the verification trace they are indistinguishable for the combiners verification algorithm. This forces the combiner to violate either unique provability or perfect correctness.

We show the description of the full adversary $\mathcal{A}$ in Figure 3.3.

Formal Treatment. Let Π_{VRF} be an arbitrary robust $(2;1)$ black-box DAG combiner for the VRF primitive. Let $\mathcal{O}_{vrf}$ be a VF oracle implementing a secure VRF and let $\mathcal{O}_{uppc}$ be an oracle implementing a trivial, uniquely provable and perfectly correct VF such that the key generation algorithm always returns $(\mathsf{sk}_{def}, \mathsf{pk}_{def})$, the evaluation algorithm returns (y_{def}, π_{def}) for every input and the verification algorithm accepts a tuple (pk, x, y, π) if and only if $y = y_{def}$. First, we show two important results that central to the proof stated in Claim 3.6 and Lemma 3.7.

We claim that we can assume without loss of generality, that the robust black-box DAG combiner for the VRF primitive, during the execution of its evaluation algorithm on arbitrary inputs, queries every oracle algorithm on any given input at most once.

Claim 3.6 *Let Π_{VRF} be a robust $(n;k)$ black-box DAG VRF-combiner in the sense of Definition 3.1. There exists a robust $(n;k)$ black-box DAG VRF-combiner $\widetilde{\Pi}_{\mathsf{VRF}}$ such that in the execution of the evaluation algorithm it queries any oracle algorithm on any given input at most once.*

Proof (Claim 3.6) Let Π_{VRF} be a robust $(n;k)$ black-box DAG VRF-combiner. We obtain $\widetilde{\Pi}_{\mathsf{VRF}}$ by modifying Π_{VRF} in the following way: At every vertex $v \in V(\mathcal{G}_{eval})$ the Turing machine $\tilde{f}^v_{eval}$ that maintains a table T_v of mappings from previous oracle queries $\mathbf{q}_w$ for $w \in \mathcal{N}^-_{\mathcal{G}_{eval}}(v)$ to oracle responses $\mathbf{x}_w$ of the same oracle algorithm $\ell_{eval}(v)$. $\widetilde{f}^v_{eval}$ then simulates the Turing machine f^v_{eval} until it returns the query $\mathbf{q}_v$. Then $\widetilde{f}^v_{eval}$ checks whether $\mathbf{q}_w$ is contained in the table T_v. If that is not the case we issue the query $\mathbf{q}_w$ to $\mathcal{O}_{\ell_{eval}}(\mathbf{q}_w)$, otherwise we issue a fresh query $\widetilde{\mathbf{q}}_w$ not in the table. If the oracle is queried on $\widetilde{\mathbf{q}}_w$ subsequent Turing machines f^u_{eval} with $(v,u) \in E(\mathcal{G}_{eval})$, will then simply find the first query to the oracle $c(v)$ with the same input if they are also incident to u. To ensure this we will again simply draw an edge from the source to every other vertex, from every vertex (d,i,j) to any vertex (d',i',j') if $d < d'$ and from every vertex to the sink.

To show that $\widetilde{\Pi}_{\mathsf{VRF}}$ is indeed a valid robust $(n;k)$ black-box DAG VRF-combiner it is sufficient to show that for any tuple of n candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$ of which at least k implement a secure VRF. The algorithms implemented by $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}$ and $\widetilde{\Pi}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}$ are equivalent. Since we dont modify the algorithms $\mathsf{Gen}_{\mathsf{VRF}}$ and $\mathsf{Vfy}_{\mathsf{VRF}}$ we only need to show that equivalence holds for $\mathsf{Eval}_{\mathsf{VRF}}$.

Assume towards a contradiction that this is not the case, i.e. that there exists an input $(\mathsf{sk}, x) \in \mathcal{SK} \times \{0,1\}^\lambda$ such that $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = y_1$ and $\widetilde{\Pi}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = y_2$ with $y_1 \neq y_2$. Since the Turing machine around

the oracle calls are simulated the same way in both this either means that f^t_{eval} is probabilistic which or that an oracle query is answered differently and thus not deterministic. In both cases we have that both y_1, y_2 could also have been produced by $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}$. By perfect correctness both y_1, y_2 must be verifiable but by unique provability only at most one of y_1, y_2 can be verifiable. A contradiction. We conclude that therefore the algorithm $\mathsf{Eval}_{\mathsf{VRF}}$ of $\Pi^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}$ and $\widetilde{\Pi}^{\mathcal{O}_1,\ldots,\mathcal{O}_2}_{\mathsf{VRF}}$ are functionally equivalent.

To show a polynomial runtime bound on $\widetilde{\Pi}^{\mathcal{O}_1,\ldots,\mathcal{O}_n}_{\mathsf{VRF}}$ we □ …

From now on we will assume that for every input to the combiners evaluation algorithm, and oracle is queried on any given input at most once. That is, if we remap the output of an oracle on a single point, then at most one oracle query differs in the execution of the evaluation algorithm for every input.

Next, let $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ and $\widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ be sets with size bounded by a polynomial, i.e. $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$. We can construct an oracle $\widetilde{\mathcal{O}}_{uppc}$ from $\mathcal{O}_{vrf}$ and $\mathcal{O}_{uppc}$ such that $\widetilde{\mathcal{O}}_{uppc}$ implements a secure VRF and $\widetilde{\mathcal{O}}_{uppc}$ agrees with $\mathcal{O}_{uppc}$ on the outputs of the evaluation algorithm on all points $\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}$ and on the verification algorithm on all points $\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}$. Formally stated in the following lemma.

Lemma 3.7 *Let $\mathcal{O}_{vrf}$ be a VRF oracle in the sense of Definition 2.9 and $\mathcal{O}_{uppc}$ a VF oracle that is perfectly correct in the sense of Definition 2.4 and uniquely provable in the sense of Definition 2.5. Let $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ and $\widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ be polynomially bounded sets, i.e. $|\widetilde{\mathcal{SK}}|, |\widetilde{\mathcal{PK}}| \in \mathsf{poly}(\lambda)$. There exists an oracle $\widetilde{\mathcal{O}}_{uppc}$ satisfying the following properties.*

1. *The evaluation algorithm of $\widetilde{\mathcal{O}}_{uppc}$ agrees with the evaluation algorithm of $\mathcal{O}_{uppc}$ on all secret keys $\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}, \widetilde{\mathcal{O}}_{uppc}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot) = \mathcal{O}_{uppc}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, \cdot)$$

2. *The verification algorithm of $\widetilde{\mathcal{O}}_{uppc}$ agrees with the evaluation algorithm of $\widetilde{\mathcal{O}}_{uppc}$ on all public keys $\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}$, i.e. we have that*

$$\forall \widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}, \widetilde{\mathcal{O}}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot) = \mathcal{O}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, \cdot, \cdot, \cdot)$$

3. *The oracle $\widetilde{\mathcal{O}}_{uppc}$ implements a secure VRF in the sense of Definition 2.9.*

A too simple idea would be to replace the mapping of $\mathcal{O}_{vrf}.\mathsf{Eval}_{\mathsf{VRF}}$ on all inputs (sk, x) with $\mathsf{sk} \in \widetilde{\mathcal{SK}}$ and similarly modify the mapping of $\mathcal{O}_{vrf}.\mathsf{Vfy}_{\mathsf{VRF}}$ on all inputs (pk, x, y, π) with $\mathsf{pk} \in \widetilde{\mathcal{PK}}$. However, the resulting oracle doesn't necessarily need to be uniquely provable or perfectly correct anymore. For

instance, imagine the scenario where a secret key $\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}$ is remapped but the corresponding public key pk does not itself belong to $\widetilde{\mathcal{PK}}$. In that case the verification algorithm on pk is left unchanged while the evaluation algortihm on $\widetilde{\mathsf{sk}}$ is modified. This can result in correctly computed image-proof pairs to be rejected by the verification algorithm, violating perfect correctness.

Proof (Lemma 3.7) We begin by introducing a two relations $\sim_{\mathcal{SK}} \subseteq \mathcal{SK}^2$ and $\sim_{\mathcal{PK}} \subseteq \mathcal{PK}^2$. Intuitively, these relations capture whether two secret or public keys have a counterpart which could have been sampled with both keys. Formally we define the relations $\sim_{\mathcal{SK}}, \sim_{\mathcal{PK}}$ as follows:

$$\mathsf{sk}_1 \sim_{\mathcal{SK}} \mathsf{sk}_2 \iff \exists \mathsf{pk} \in \mathcal{PK}, \Pr\left[\begin{matrix}(\mathsf{sk}_1, \mathsf{pk}) \leftarrow_{\$} \mathcal{O}_{uppc}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}) \leftarrow_{\$} \mathcal{O}_{uppc}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{matrix}\right] > 0 \tag{3.5}$$

$$\mathsf{pk}_1 \sim_{\mathcal{PK}} \mathsf{pk}_2 \iff \vee \exists \mathsf{sk} \in \mathcal{SK}, \Pr\left[\begin{matrix}(\mathsf{sk}, \mathsf{pk}_1) \leftarrow_{\$} \mathcal{O}_{uppc}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda) \wedge \\ (\mathsf{sk}, \mathsf{pk}_2) \leftarrow_{\$} \mathcal{O}_{uppc}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\end{matrix}\right] > 0 \tag{3.6}$$

We now consider the reflexive transitive closure of those relations $\sim^*_{\mathcal{SK}}, \sim^*_{\mathcal{PK}}$, formally defined by $\sim^* = \bigcup_{k \in \mathbb{N}} \sim^k$. Since $\sim_{\mathcal{SK}}$ and $\sim_{\mathcal{PK}}$ are symmetric by definition, their reflexive transitive closures are reflexive, symmetric and transitive and hence equivalence relations. Given an element sk we denote its equivalence class under $\sim^*_{\mathcal{SK}}$ with $[\mathsf{sk}]_{\sim^*_{\mathcal{SK}}} = \{\mathsf{sk}' \in \mathcal{SK} : \mathsf{sk}' \sim^*_{\mathcal{SK}} \mathsf{sk}\}$, analogously for an element pk and the relation $\sim^*_{\mathcal{PK}}$ we define $[\mathsf{pk}]_{\sim^*_{\mathcal{PK}}} = \{\mathsf{pk}' \in \mathcal{PK} : \mathsf{pk}' \sim^*_{\mathcal{PK}} \mathsf{pk}\}$.

Additionally, for secret and public keys we define the function $R_{\mathcal{SK}}$ and $R_{\mathcal{PK}}$ respectively that map a key to all counterparts which are sampled with that key with non-negligible probability.

$$R_{\mathcal{PK}}(\mathsf{pk}) = \left\{\mathsf{sk} \in \mathcal{SK} : \Pr\left[(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.7}$$

$$R_{\mathcal{SK}}(\mathsf{sk}) = \left\{\mathsf{pk} \in \mathcal{PK} : \Pr\left[(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] > 0\right\} \tag{3.8}$$

Next, we define the notion of the closure of a set of secret or public keys. Given a subset of secret keys $\widetilde{\mathcal{SK}}$ or public keys $\widetilde{\mathcal{PK}}$ we define its closure $\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ and $\mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})$ which are just the union over the equivalence classes of all keys in the subsets. We formally define the functions as follows:

$$\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}) = \bigcup_{\widetilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}} [\widetilde{\mathsf{sk}}]_{\sim^*_{\mathcal{SK}}} \tag{3.9}$$

$$\mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}}) = \bigcup_{\widetilde{\mathsf{pk}} \in \widetilde{\mathcal{PK}}} [\widetilde{\mathsf{pk}}]_{\sim^*_{\mathcal{PK}}} \tag{3.10}$$

Observe that the secret key closure $\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ for a subset $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ is the set of secret keys that need to be remapped if we want to remap all of the evaluations on the secret keys $\widetilde{\mathcal{SK}}$ to preserve unique provability and correctness. The same holds for the public key closure. Now we define the closure for subsets $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ and $\widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ at the same time. They correspond to the minimal set of secret or public keys that need to be remapped to preserved unique provability and perfect correctness of $\mathcal{O}_{uppc}$ if we want to remap all keys in $\widetilde{\mathcal{SK}}$ and $\widetilde{\mathcal{PK}}$.

$$\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) = \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}) \cup \bigcup_{\widetilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\widetilde{\mathsf{pk}}) \tag{3.11}$$

$$\mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) = \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}}) \cup \bigcup_{\widetilde{\mathsf{sk}} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})} R_{\mathcal{SK}}(\widetilde{\mathsf{sk}}) \tag{3.12}$$

Finally, we constuct an oracle $\widetilde{\mathcal{O}}_{uppc}$ that interpolates between $\mathcal{O}_{uppc}$ and $\mathcal{O}_{vrf}$. The key generation algorithm is defined to be the function for key generation of $\mathcal{O}_{vrf}$, the evaluation and the verification algorithm of $\widetilde{\mathcal{O}}_{uppc}$ are defined as follow, they evaluate to the output of $\mathcal{O}_{uppc}$ on all keys in $\widetilde{\mathcal{SK}}$ and $\widetilde{\mathcal{PK}}$ and they evaluate to the output of $\mathcal{O}_{vrf}$ otherwise.

$$\widetilde{\mathcal{O}}_{uppc}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) = \begin{cases} \mathcal{O}_{uppc}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{if } \mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) \\ \mathcal{O}_{vrf}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x) & \text{otherwise} \end{cases} \tag{3.13}$$

$$\widetilde{\mathcal{O}}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = \begin{cases} \mathcal{O}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{if } \mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) \\ \mathcal{O}_{vrf}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) & \text{otherwise} \end{cases} \tag{3.14}$$

The properties 1. and 2. of Lemma 3.7 hold by definition of $\widetilde{\mathcal{O}}_{uppc}$ since $\widetilde{\mathcal{SK}} \subseteq \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}) \subseteq \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ and $\widetilde{\mathcal{PK}} \subseteq \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}}) \subseteq \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ which hold by reflexivity of $\sim^*_{\mathcal{SK}}$ and $\sim^*_{\mathcal{PK}}$ respectively. We show that 3. holds by showing the $\widetilde{\mathcal{O}}_{uppc}$ is uniquely provable, perfectly correct and pseudorandom.

Claim 3.8 *Let $\mathcal{O}_{uppc}$ be a VF oracle that is uniquely provable in the sense of Definition 2.5 and perfectly correct in the sense of Definition 2.4 and let $\mathcal{O}_{vrf}$ be a VF oracle that implements a secure VRF in the sense of Definition 2.9. The oracle $\widetilde{\mathcal{O}}_{uppc}$ constructed from $\mathcal{O}_{up}$ and $\mathcal{O}_{vrf}$ for given subsets $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ and $\widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ as in* (3.13) *and* (3.14) *is perfectly correct in the sense of Definition 2.4 and uniquely provable in the sense of Definition 2.5.*

Proof (Claim 3.8) Let $\lambda \in \mathbb{N}$ be arbitrary but fixed. We show unique provability and perfect correctness seperately.

Correctness: Let $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \widetilde{\mathcal{O}}_{uppc}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$, let $x \in \{0,1\}^\lambda$ be arbitrary and let $(y, \pi) \leftarrow \widetilde{\mathcal{O}}_{uppc}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$. To show that $\widetilde{\mathcal{O}}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi) = 1$ with probability 1 it suffices to show that $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ if and only if $\mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$, since if $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ and hence $\mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ verifies by perfect correctness of $\mathcal{O}_{uppc}$ while if $\mathsf{sk} \notin_{\mathcal{SK}} (\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ and hence $\mathsf{pk} \notin \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ verifies by perfect correctness of $\mathcal{O}_{vrf}$.

We show $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) \implies \mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$, the converse follows analogously. Assume the left-hand side, i.e. $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ and distinguish two cases based on the definition of $\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$. If $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ then since $\mathsf{pk} \in R_{\mathcal{SK}}(\mathsf{sk})$ we obtain $\mathsf{pk} \in \bigcup_{\tilde{\mathsf{sk}} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})} R_{\mathcal{SK}}(\tilde{\mathsf{sk}})$ and hence $\mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$. On the other hand, if $\mathsf{sk} \in \bigcup_{\tilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\tilde{\mathsf{pk}})$ then $\mathsf{sk} \in R_{\mathcal{PK}}(\tilde{\mathsf{pk}})$ for some $\tilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})$. By definition of the closure we have $\mathsf{pk} \sim^*_{\mathcal{PK}} \tilde{\mathsf{pk}}$, hence $\mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})$ and thus $\mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$. more details

Unique Provability: Assume towards a contradiction that there exist two image-proof pairs (y_1, π_1) and (y_2, π_2) with distinct images $y_1 \neq y_2$ such that $\widetilde{\mathcal{O}}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_1, \pi_1) = 1$ and $\widetilde{\mathcal{O}}_{uppc}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y_2, \pi_2) = 1$. If $\mathsf{pk} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ then verification is defined by $\mathcal{O}_{uppc}$ and we arrive at a contradiction with the unique provability of $\mathcal{O}_{uppc}$. If $\mathsf{pk} \notin \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ then verification is defined by $\mathcal{O}_{vrf}$ and therefore we arrive at a contradiction with the assumed unique provability of $\mathcal{O}_{vrf}$. In either case we reach a contradiction, hence there cannot exist such image-proof pairs.

We conclude that $\widetilde{\mathcal{O}}_{uppc}$ is perfectly correct and uniquely provable. □

Claim 3.9 *Let $\mathcal{O}_{uppc}$ be a VF oracle that is uniquely provable in the sense of Definition 2.5 and perfectly correct in the sense of Definition 2.4 and let $\mathcal{O}_{vrf}$ be a VF oracle that implements a secure VRF in the sense of Definition 2.9 that is $(t^{prf}_{vrf}, \varepsilon^{prf}_{vrf})$-pseudorandom. The oracle $\widetilde{\mathcal{O}}_{uppc}$ constructed from $\mathcal{O}_{up}$ and $\mathcal{O}_{vrf}$ for given subsets $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}$ and $\widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ as in* (3.13) *and* (3.14) *is $(t^{prf}_{uppc}, \varepsilon^{prf}_{uppc})$-pseudorandom in the sense of Definition 2.6. Where*

$$\varepsilon^{prf}_{vrf} = \left(1 + \left|\widetilde{\mathcal{SK}}\right| + \left|\widetilde{\mathcal{PK}}\right|\right)\varepsilon^{prf}_{uppc} \quad \textit{and}$$

Use a hybrid argument for the proof. Intuitively the proof works by bounding the probability that a bad event $\mathcal{E}$, that the secret key sk^* sampled in the pseudorandomness security experiment is in the set $\mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$. If $\mathcal{E}$ doesn't occur, then the security game is identical to the security game against $\mathcal{O}_{vrf}$ which no adversary running in time t^{prf}_{vrf} wins with advantage greater than ε^{prf}_{vrf}. Using the difference lemma we can then bound the advantage of any adversary playing the pseudorandomness game against $\widetilde{\mathcal{O}}_{uppc}$ from above by an expression in ε^{prf}_{vrf}. Before we formally prove Claim 3.9 we state the difference lemma, we defer the proof of it to the appendix since its a known result.

Lemma 3.10 (Difference Lemma) *let W_0, W_1 and B be events satisfying:*

$$\Pr[W_0 \mid \neg B] = \Pr[W_1 \mid \neg B]$$

Then we can impose the following bound:

$$|\Pr[W_0] - \Pr[W_1]| \leq \Pr[B]$$

Proof (Claim 3.9) We define games H_1, H_2, H_3 as follows.

- H_1 is the simple pseudorandomness experiment for the hidden bit $b = 0$ against the oracle $\mathcal{O}_{vrf}$, i.e. the adversary is playing against the prf security game where he is given a challenge image y^* computed from the challenge pre-image x^* using $\mathcal{O}_{vrf}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}^*, x^*)$ where sk^* is the hidden secret key sampled by the challenger.
- H_2 is the simple pseudorandomness experiment for the hidden bit $b = 0$ against the oracle $\widetilde{\mathcal{O}}_{uppc}$, i.e. the adversary is playing the prf security game where he is given a challenge image y^* computed from the challenge pre-image x^* using $\widetilde{\mathcal{O}}_{uppc}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}^*, x^*)$ where sk^* is the hidden secret key sampled by the challenger.
- H_3 is the simple pseudorandomness experiment for the hidden bit $b = 1$ against the oracle $\mathcal{O}_{vrf}$, i.e. the adversary is playing the prf security game where he is given a challenge image y^* which the challenger sampled u.a.r. from the image co-domain $\mathcal{Y}$.

We write $H_i^{\lambda,\mathcal{A}} = 1$ to denote the event that adversary $\mathcal{A}$ wins the game H_i for all $i \in [3]$, we can now bound the advantage of any adversary $\mathcal{A}$ against $\widetilde{\mathcal{O}}_{uppc}$ in the pseudorandomness security experiment as follows.

$$\begin{aligned}
\mathbf{Adv}^{\mathrm{prf}}_{\widetilde{\mathcal{O}}_{uppc}}(\lambda, \mathcal{A}) &= \left|\Pr\left[H_2^{\lambda,\mathcal{A}} = 1\right] - \Pr\left[H_3^{\lambda,\mathcal{A}} = 1\right]\right| && (3.15)\\
&\leq \left|\Pr\left[H_2^{\lambda,\mathcal{A}} = 1\right] - \Pr\left[H_1^{\lambda,\mathcal{A}} = 1\right]\right| && (3.16)\\
&\quad + \left|\Pr\left[H_1^{\lambda,\mathcal{A}} = 1\right] - \Pr\left[H_3^{\lambda,\mathcal{A}} = 1\right]\right| && (3.17)\\
&\leq \left|\Pr\left[H_2^{\lambda,\mathcal{A}} = 1\right] - \Pr\left[H_1^{\lambda,\mathcal{A}} = 1\right]\right| + \mathbf{Adv}^{\mathrm{prf}}_{\mathcal{O}_{vrf}}(\lambda, \mathcal{A}) && (3.18)\\
&\leq \left|\Pr\left[H_2^{\lambda,\mathcal{A}} = 1\right] - \Pr\left[H_1^{\lambda,\mathcal{A}} = 1\right]\right| + \varepsilon^{prf}_{vrf} && (3.19)
\end{aligned}$$

Consider the event that the challenger in the pseudorandomness security experiment samples a secret key $\mathsf{sk}^* \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$. If $\mathsf{sk}^* \notin \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$ then in H_2 we are playing the prf security game against $\widetilde{\mathcal{O}}_{uppc}$ which is evaluates to the same outputs as $\mathcal{O}_{vrf}$. Hence we have that under the event not happening that $H_1 = H_2$ and therefore in particular that

maybe arrive here seperately

$$\Pr\left[H_2^{\lambda,\mathcal{A}} = 1\right] = \Pr\left[H_1^{\lambda,\mathcal{A}} = 1\right]$$

It remains to bound the probability that the event $\mathsf{sk}^* \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}})$.

Claim 3.11 *Let $\widetilde{\mathcal{SK}} \subseteq \mathcal{SK}, \widetilde{\mathcal{PK}} \subseteq \mathcal{PK}$ be polynomially bounded sets, then*

$$\Pr\left[\mathsf{sk}^* \in cl_{\mathcal{SK}}(\widetilde{\mathcal{SK}}, \widetilde{\mathcal{PK}}) : (\mathsf{sk}^*, \mathsf{pk}^*) \leftarrow_{\$} \mathcal{O}_{vrf}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)\right] \leq \left(\left|\widetilde{\mathcal{SK}}\right| + \left|\widetilde{\mathcal{PK}}\right|\right)\varepsilon_{vrf}^{prf}$$

Applying Lemma .1 and subsequently Claim 3.11 to Equation (3.19):

$$\mathbf{Adv}^{\mathrm{prf}}_{\tilde{\mathcal{O}}_{uppc}}(\lambda, \mathcal{A}) \leq (1 + |\widetilde{\mathcal{SK}}| + |\widetilde{\mathcal{PK}}|)\varepsilon_{vrf}^{prf} \tag{3.20}$$

□ ...

We bound the probabilities and arrive at the final bound we show that if the probability were non-negligible an adversary would win the pseudorandomness security experiment against $\mathcal{O}_{\mathbf{VRF}}$ with non-negligible advantage, which is a contradiction since we assume $\mathcal{O}_{\mathbf{VRF}}$ to be pseudorandom. For the subsequent proofs keep in mind that the probabilities are over the randomly sampled secret key. We consider both probabilities seperately,

For the bound on $\Pr[\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})]$ note that for any value of x the set of possible images is given by $\{y \leftarrow_{\$} \mathcal{O}_{up}.\mathsf{Eval}_{\mathsf{VRF}}(\tilde{\mathsf{sk}}, x) : \tilde{\mathsf{sk}} \in \widetilde{\mathcal{SK}}\}$ which is bounded in cardinality from above by $\widetilde{\mathcal{SK}}$. Recall that $|\widetilde{\mathcal{SK}}| \in \mathsf{poly}(\lambda)$. Conditioned on the event that $\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})$ there exists an adversary $\mathcal{A}$ with advantage at least $1/|\widetilde{\mathcal{SK}}|$. Hence we can bound $\Pr[\mathsf{sk} \in \mathsf{cl}_{\mathcal{SK}}(\widetilde{\mathcal{SK}})]$ from above by $\varepsilon^{prf}_{VRF,mix}|\widetilde{\mathcal{SK}}|$.

For the bound on $\Pr[\mathsf{sk} \in \bigcup_{\tilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\tilde{\mathsf{pk}})]$ by a similar argument the mappings also correspond to elements of $\widetilde{\mathcal{PK}}$. Again, following this line of argument we condition on the event that $\mathsf{sk} \in \bigcup_{\tilde{\mathsf{pk}} \in \mathsf{cl}_{\mathcal{PK}}(\widetilde{\mathcal{PK}})} R_{\mathcal{PK}}(\tilde{\mathsf{pk}})$ and find an adversary with advantage bounded from below by $1/|\widetilde{\mathcal{PK}}|$. We obtain the bound on the probability of $\varepsilon^{prf}_{\mathbf{VRF}}|\widetilde{\mathcal{PK}}|$. □

Equipped with Lemma 3.7, we now consider the combiner $\Pi^{\mathcal{O}_{vrf},\mathcal{O}_{up}}_{\mathsf{VRF}}$ initialized with the oracles $\mathcal{O}_{vrf}$ and $\mathcal{O}_{up}$ as introduced above. Fix a key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \Pi^{\mathcal{O}_{vrf},\mathcal{O}_{up}}_{\mathsf{VRF}}$, an evaluation point $x \in \{0,1\}^\lambda$, the corresponding output $\leftarrow \Pi^{\mathcal{O}_{vrf}\mathcal{O}_{up}}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$ and the verification trace $\tau \leftarrow \mathsf{trace}(\Pi^{\mathcal{O}_{vrf},\mathcal{O}_{up}}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi))$.

We classify the vertices of $V(\mathcal{G}_{eval})$ as follows. A vertex $v \in V(G_{eval})$ is called *exposed* if the oracle query issued at v during the evaluation of the input (sk, x) also appears in the verification trace τ. Otherwise, we call v *obscure*. Observe that exposed queries and queries to $\mathcal{O}_{uppc}$ can be answered without calling any oracle by looking at the verification trace or taking the default values $\mathsf{sk}_{def}, \mathsf{pk}_{def}, y_{def}, \pi_{def}$ respectively . The only queries whose responses are not directly available without any oracle queries are the obscure queries to $\mathcal{O}_{vrf}$. We claim that these responses can be substituted with arbitrary, but well-formed, oracle responses without affecting the combiners output (sk, x).

verification easy by seeing if image y_{def}

Claim 3.12 *Let $\mathcal{O}_{vrf}$ and $\mathcal{O}_{uppc}$ be defined as above, let $\mathsf{sk}, \mathsf{pk}, x, y, \pi, \tau$ be defined as above and let $V_{obs} \subseteq V(\mathcal{G}_{eval})$ be the set of obscure vertices to $\mathcal{O}_{vrf}$. Then the oracles responses at vertices in V_{obs} are irrelevant in the sense that for any assignment of well-formed responses to the queries at vertices in V_{obs} the algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}$ produces the same image y.*

Proof (Claim 3.12) Assume this were not the case, let $\widehat{\mathcal{O}}_{vrf}$ and $\widetilde{\mathcal{O}}_{vrf}$ be the oracles obtained from $\mathcal{O}_{vrf}$ by modifying the mapping on the obscure vertices V_{obs} to the two assignments to obtain two different images $y_0 \neq y_1$ and let $\widetilde{\mathcal{O}}_{uppc}$ be the VRF oracle obtained from $\mathcal{O}_{uppc}$ by applying the procedure detailed in Lemma 3.7. Since $\widetilde{\mathcal{O}}_{uppc}$ is a secure VRF, the combiner initialized with the oracles $\widehat{\mathcal{O}}_{vrf}, \widetilde{\mathcal{O}}_{uppc}$ and the combiner initialized with the oracle $\widetilde{\mathcal{O}}_{vrf}, \widetilde{\mathcal{O}}_{uppc}$ must implement secure VRFs and in particular also uniquely provable and perfectly correct VFs. Further note that by Claim 3.6 we note that there can be no conflict of having to assign two different outputs to the query made to the same oracle on the same input at a different position since every oracle algorithm is queried at any input at most once. Hence:

$$\Pi_{\mathsf{VRF}}^{\widetilde{\mathcal{O}}_{vrf},\widetilde{\mathcal{O}}_{uppc}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \tilde{y}, \tilde{\pi}) = 1 \tag{3.21}$$

$$\Pi_{\mathsf{VRF}}^{\widehat{\mathcal{O}}_{vrf},\widetilde{\mathcal{O}}_{uppc}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, \tilde{y}, \tilde{\pi}) = 0 \tag{3.22}$$

Since the oracle $\widehat{\mathcal{O}}_{vrf}$ and $\widetilde{\mathcal{O}}_{vrf}$ only differ on obscure queries we have that the verification algorithm on $(\mathsf{pk}, x, \tilde{y}, \tilde{\pi})$ must evaluate the same and to the same result. This is a contradiction with (3.21) and (3.22). □

We define the evaluation simulation $\mathsf{EvalSim}_{\mathsf{VRF}}$ that simulates the evaluation without invoking any oracle queries. The simulation will simply traverse the evaluation algorithm of the DAG combiner. For any oracle query it checks the corresponding transcript τ_i. If the query is either to oracle $\mathcal{O}_1$ we can honestly answer it with $\mathbf{x}_v^{def}$ or in the transcript, then we can answer it hoenstly as well. If the query is *obscure* then we branch once for every possible assignment of the oracle response which fits the given format, e.g. for a query to eval we branch for every answer $(y, \pi) \in \mathcal{Y} \times \mathcal{P}$. The pseudocode for $\mathsf{EvalSim}_{\mathsf{VRF}}$ is shown in Figure 3.2. In the following we show that this algorithm satisfies a few properties. We want to note that the evaluation simulation algorithm and is clearly inefficient since it enumerates possibly many well-formed oracle queries, since the adversary will use it only in combination with the PSPACE complete oracle in the end that is not a problem.

Algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau)$

$v_1, \ldots, v_m \leftarrow$ topological ordering of $V(\mathcal{G}_{eval}) \setminus \{s_{eval}, t_{eval}\}$
$c_{s_{eval}} \leftarrow 1$ // Number of states or branches
for $i = 1$ to m **do**
  // Reconstruct all branches:
  $\Sigma \leftarrow \varnothing$
  for $w \in \mathcal{N}^-_{\mathcal{G}_{eval}}(v_i)$ **do**
    $\Sigma \leftarrow \{\Sigma \cup \{(w, \mathbf{x}_w^{(j)})\} : j \in [c_w]\}$
  // Iterate all branches:
  $j \leftarrow 1$
  for $\sigma \in \Sigma$ **do**
    $\mathbf{q}_{v_i}^{(j)} \leftarrow f_{eval}^{v_i}(\sigma)$
    if $\ell(v_i) = (2, _)$ **then** // Query to oracle $\mathcal{O}_{uppc}$
      $\mathbf{x}_{v_i}^{(j)} \leftarrow \mathbf{x}_{v_i}^{def}$
    else if $\mathbf{q}_{v_i}^{(j)} \in \tau$ **then** // Exposed query to $\mathcal{O}_{vrf}$
      $\mathbf{x}_{v_i}^{(j)} \leftarrow$ response to $\mathbf{q}_{v_i}^{(j)}$ in τ
    else if $\ell(v_i) = (1, _)$ **then** // Obscure query to $\mathcal{O}_{vrf}$
      for $\mathbf{s} \in \mathcal{S}_{v_i}$ **do**
        $\mathbf{x}_{v_i}^{(j)} \leftarrow \mathbf{s}$
        $j \leftarrow j + 1$
    $c_{v_i} \leftarrow j$
// Check if all evaluation branches result in y
  $\Sigma \leftarrow \varnothing$
  for $w \in \mathcal{N}^-_{\mathcal{G}_{eval}}(t_{eval})$ **do**
    $\Sigma \leftarrow \{\Sigma \cup \{(w, \mathbf{x}_w^{(j)})\} : j \in [c_w]\}$
  for $\sigma \in \Sigma$ **do**
    $\mathbf{y}_{t_{eval}} \leftarrow f_{eval}^{t_{eval}}(\sigma)$
    if $\mathbf{y}_{t_{eval}} \neq y$ **then**
      return $\perp$
return $\top$

Figure 3.2: Pseudocode of the algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}$. We use c_v to denote the number of possible branches at vertex v, $\mathbf{x}_v^{def}$ for the default value of the oracle of $\mathcal{O}_{uppc}$ at vertex v (e.g. if $c(v) = (2,2)$ then (y_{def}, π_{def})) and with $\mathcal{S}_v$ the corresponding co-domain of the oracle at v (e.g. if $c(v) = (1,2)$ then $\mathcal{Y} \times \mathcal{P}$).

Claim 3.13 *Let* $\mathsf{EvalSim}_{\mathsf{VRF}}$ *be the algorithm as described above and in Figure 3.2 taking as input a tuple* $(\mathsf{sk}, x, y, \pi, \tau)$ *containing a secret key, evaluation point, image, proof and a trace of the combiners verification algorithm on* (pk, x, y, π) *where* pk *is the true public key. Then* $\mathsf{EvalSim}_{\mathsf{VRF}}$ *satisfies the following properties.*

1. $\mathsf{EvalSim}_{\mathsf{VRF}}$ *makes no oracle queries to the candidates* $\mathcal{O}_{vrf}, \mathcal{O}_{up}$
2. *If* $\mathsf{sk} = \mathsf{sk}^*$ *then* $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \top$
3. *If* $\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x) \neq y$ *then* $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) = \bot$

Informally, this means that the evaluation simulation will return with $\top$ on a non-empty subset of secret keys which agrees with the true secret key sk^* on the evaluation of x. Non-empty because the security experiments secret key sk^* will always result in $\top$ due to the second property.

Proof (Claim 3.13) The first property holds by construction. The second property follows directly from Claim 3.12. The third property is because the true execution of the combiners evaluation algorithm corresponds is always a branch in the execution of $\mathsf{EvalSim}_{\mathsf{VRF}}$, if the output of that branch is not y then by construction the combiner will return $\bot$. □

Using this simulator $\mathsf{EvalSim}_{\mathsf{VRF}}$ we can describe the set of iteratively filtered candidate secret keys as follows. Let $\{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]}$ be a set of $s(\lambda) + \lambda$ tuples containing a uniformly random evaluation point $x_i \in \{0,1\}^\lambda$, the evaluation of x_i resulting in the tuple (y_i, π_i) and the trace of the combiners verification algorithm on (x_i, y_i, π_i). The set of candidate keys is defined as:

$$\mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i) := \{\mathsf{sk} \in \mathcal{SK} : \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i)\}$$

We claim that after filtering for $s(\lambda) + \lambda$ rounds, the probability that a remaining candidate key sk_{cand} sampled u.a.r. from the set of remaining candidate keys evaluates a uniformly random evaluation point $x^* \in \{0,1\}^*$ to the same image y^* as the games secret key sk^* does is non-negligible.

Claim 3.14 *Let* $\mathcal{SK}_{cand}$ *be defined as above and let* $\{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]}$ *then we have for any* sk^* *sampled u.a.r. from the set of filtered candidate keys, i.e.*

$$\mathsf{sk}^* \leftarrow_{\$} \bigcap_{i\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)$$

that the probability of a uniformly random challenge pre-image $x^* \in \{0,1\}^\lambda \setminus \{x_1, \ldots, x_{s(\lambda)+\lambda}\}$ *the black-box combiner instanciated with the oracles* $\mathcal{O}_{vrf}, \mathcal{O}_{uppc}$ *evaluates to the same image* y^* *as for the secret key* sk^* *is non-negligible, i.e.*

$$\Pr\left[\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}^*, x) = \Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}^{y}_{\mathsf{VRF}}(\mathsf{sk}, x)\right] \geq \frac{1}{p(\lambda)}$$

For p *a polynomial in* λ.

Proof (Claim 3.14) For a secret key $\mathsf{sk} \in \mathcal{SK}$ we define $\mathcal{X}(\mathsf{sk})$ to be the number of evaluation points $x \in \{0,1\}^\lambda$ for which the evaluation of x under sk agrees with the evaluation of x under sk^*, the games secret key, i.e.

$$\mathcal{X}(\mathsf{sk}) = \left|\left\{x \in \{0,1\}^\lambda : \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x) = \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}^*, x)\right\}\right|$$

We say that a secret key sk is *unfavorable*, if the probability of an evaluation point x sampled u.a.r. from $\{0,1\}^\lambda$ evaluating to a different image under sk then under sk^* is at least $1/2$. Let $\mathcal{SK}_{unfav} \subseteq \mathcal{SK}$ be the set of unfavorable secret keys. Note that the probability of a secret key sk surviving a filtering step is given by the following.

$$\Pr[\mathsf{sk} \in \mathcal{SK}_{cand}(x, y, \pi, \tau)] = \frac{\mathcal{X}(\mathsf{sk})}{2^\lambda}$$

We can bound the expected number of unfavorable secret keys that survive $s(\lambda) + \lambda$ rounds of secret key filtering over the randomness of $x_1, \ldots, x_{s(\lambda)+\lambda}$. We use $\mathbf{1}_{\mathcal{E}}$ to denote the random indicator variable for the event $\mathcal{E}$.

$$\mathbb{E}\left[\left|\mathcal{SK}_{unfav} \cap \bigcap_{i\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right|\right] \tag{3.23}$$

$$= \mathbb{E}\left[\sum_{\mathsf{sk}_{unfav}\in\mathcal{SK}_{unfav}} \mathbf{1}_{\mathsf{sk}_{unfav}\in\bigcap_{i\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_i,y_i,\pi_i,\tau_i)}\right] \tag{3.24}$$

$$= \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \Pr\left[\mathsf{sk}_{unfav} \in \bigcap_{i\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right] \tag{3.25}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \prod_{i\in[s(\lambda)+\lambda]} \Pr\left[\mathsf{sk}_{unfav} \in \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)\right] \tag{3.26}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \prod_{i\in[s(\lambda)+\lambda]} \frac{\mathcal{X}(\mathsf{sk}_{unfav})}{2^\lambda} \tag{3.27}$$

$$\leq \sum_{\mathsf{sk}_{unvaf}\in\mathcal{SK}_{unfav}} \prod_{i\in[s(\lambda)+\lambda]} \frac{1}{2} \tag{3.28}$$

$$\leq \frac{1}{2^\lambda} \tag{3.29}$$

Where in (3.25) we use linearity of expectation and the fact that $\mathbf{1}_{\mathcal{E}}$ are Barnoulli random variables and in (3.26) we use that the evaluation points $x_1, \ldots, x_{s(\lambda)+\lambda}$ are sampled i.i.d. from $\{0,1\}^\lambda$. We can now use a simple Markov bound to show that the probability of a unfavorable key surviving the filtering is negligibly small. Note that the expected number of surviving

unfavorable secret keys is by definition non-negative and therefore we have that the probability of all keys filtered is given by the following expression:

$$\Pr\left[\left|\mathcal{SK}_{unfav} \cap \bigcap_{i\in[s(\lambda)+\lambda]} \mathcal{SK}_{card}(x_i, y_i, \pi_i, \tau_i)\right| \geq 1\right] \leq \frac{1}{2^\lambda}$$

this section is gruesome

Define the following events:

$$\mathcal{F} := \left\{\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}^*, x^*) = \Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}.\mathsf{Eval}_{\mathsf{VRF}}^{y}(\mathsf{sk}, x^*)\right\} \tag{3.30}$$

$$\mathcal{E} := \left\{\left|\mathcal{SK}_{unfav} \cap \bigcap_{i\in[s(\lambda)+2\lambda]} \mathcal{SK}_{card}(x_i, y_i, \pi_i, \tau_i)\right| \geq 1\right\} \tag{3.31}$$

Then we have:

$$\Pr[\mathcal{F}] = \Pr[\mathcal{F} \mid \neg\mathcal{E}]\Pr[\neg\mathcal{E}] + \Pr[\mathcal{F} \mid \mathcal{E}]\Pr[\mathcal{E}] \tag{3.32}$$

$$\geq \Pr[\mathcal{F} \mid \neg\mathcal{E}]\Pr[\neg\mathcal{E}] \tag{3.33}$$

Which concludes this proof. □

Now, we define two languages $\mathcal{L}_{cand}, \mathcal{L}_{size}$ as follows. The language $\mathcal{L}_{cand}$ consists of tuples $(\rho, i, b, \{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]})$ containing the the bit i, the value of the bit b of the ρ'th candidate key in the set after filtering the set of secret keys with the tuples $(x_i, y_i, \pi_i, \tau_i)$ for all $i \in [s(\lambda) + \lambda]$. We introduce the notation $\mathrm{rk}_\rho(S)$ for $S \subseteq \{0,1\}^{s(\lambda)} = \mathcal{SK}$ and $\rho \in [|S|]$ to denote the ρ'th element in S in canonical ordering. The language $\mathcal{L}_{size}$ contains tuples $(m, \{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]})$ if m is the size of the set of candidate keys after filtering with the tuples $(x_i, y_i, \pi_i, \tau_i)$ for all $i \in [s(\lambda) + \lambda]$.

$$\mathcal{L}_{cand} := \left\{\left(\rho, i, b, \{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]}\right) : \mathrm{rk}_\rho\left(\bigcap_{j\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_j, y_j, \pi_j, \tau_j)\right)_i = b\right\} \tag{3.34}$$

$$\mathcal{L}_{size} := \left\{\left(m, \{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]}\right) : \left|\bigcap_{j\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_j, y_j, \pi_j, \tau_j)\right| \leq m\right\} \tag{3.35}$$

We show that $\mathcal{L}_{cand}$ and $\mathcal{L}_{size}$ are in PSPACE and therefore membership can be decided by the PSPACE complete oracle. This will allow the adversary to decide membership using its PSPACE complete oracle.

Claim 3.15 *$\mathcal{L}_{cand} \in$ PSPACE and $\mathcal{L}_{size} \in$ PSPACE.*

Proof (Claim 3.15) We show membership in PSPACE by providing deterministic Turing machines M_{cond} and M_{size} that decide $\mathcal{L}_{cond}$ and $\mathcal{L}_{size}$ respectivley such that they require space polynomial in their input. Before we provide the Turing machines M_{cond} and M_{size} we first show that $\mathsf{EvalSim}_{\mathsf{VRF}}$ can be executed in space polynomial in the size of the input.

To run $\mathsf{EvalSim}_{\mathsf{VRF}}$ in space polynomial in the input $(\mathsf{sk}, x, y, \pi, \tau)$ we simulate the evaluation algorithm implemented by the combiner DAG. First compute the topological ordering in polynomial space, then initialize a counter c and simulate the combiner answering the obscure queries with the tuple of assignments corresponding to the c'th element in the canonical on the tuple of well-formed assignments of the obscure queries. That way we only need to store the values of all intermediate results and queries of a single combiner evaluation execution, since this execution runs in polynomial time, every iteration only requires polynial space.

To show $\mathcal{L}_{cond} \in \mathsf{PSPACE}$ consider the following Turing machine M_{cond}. On input $(\rho, i, b, \{x_j, y_j, \pi_j, \tau_j\}_{j\in[s(\lambda)+\lambda]})$ M_{cond} starts by enumerating all secret keys $\mathsf{sk} \in \mathcal{SK}$ in lexicographic order. For every sk iterate over every tuple $(x_j, y_j, \pi_j, \tau_j)$ over $j \in [s(\lambda) + \lambda]$ and run $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_j, y_j, \pi_j, \tau_j)$. If the invocation returns $\perp$ discard the secret key and move onto the next onw. Otherwise after we iterated over all $s(\lambda) + \lambda$ tuples increase a counter c by one until $c = \rho$. If $c = \rho$ and $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_j, y_j, \pi_j, \tau_j) = \top$ then look at the i'th bit b' and accept if and only if $b' = b$.

To show $\mathcal{L}_{size} \in \mathsf{PSPACE}$ consider the following Turing machine M_{size}. On input $(m, \{x_j, y_j, \pi_j, \tau_j\}_{j\in[s(\lambda)+\lambda]})$ M_{cond} proceeds in a similar fashion to M_{cond}. M_{soze} starts by enumerating all secret keys $\mathsf{sk} \in \mathcal{SK}$ in lexicographic order, for every sk we iterate over the tuples $(x_j, y_j, \pi_j, \tau_j)$ for $j \in [s(\lambda) + \lambda]$ and run and run $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_j, y_j, \pi_j, \tau_j)$. If the invocation returns $\perp$ discard the secret key and move onto the next one. Otherwise after we iterated over all $s(\lambda) + \lambda$ tuples increase a counter c by one. After all secret keys return c. □

We now describe the adversary $\mathcal{A}$ that wins the pseudorandomness security experiment against the combiner initialized with oracles $\mathcal{O}_{vrf}$ and $\mathcal{O}_{uppc}$ with non-negligible advantage. First, $\mathcal{A}$ first samples $s(\lambda) + \lambda$ evaluation points uniformly at random $x_1, \ldots, x_{s(\lambda)+\lambda}$. Subsequently, $\mathcal{A}$ uses the evaluation oracle in the prf game to evaluate the points to obtain $(y_1, \pi_1), \ldots, (y_{s(\lambda)+\lambda}, \pi_{s(\lambda)+\lambda})$. Using the public key pk^* the adversary will then compute the combiners verifier traces for all tuples consisting of evaluation point and image-proof pair $(x_1, y_1, \pi_1), \ldots, (x_{s(\lambda)+\lambda}, y_{s(\lambda)+\lambda}, \pi_{s(\lambda)+\lambda})$ to obtain $\tau_1, \ldots, \tau_{s(\lambda)+\lambda}$. Next, the adversary will use binary search over m to find the size of the candidate key set, k, by querying the PSPACE oracle to decide membership in $\mathcal{L}_{size}$ before sampling a random element $\rho \leftarrow_{\$} [k]$ and recovering the ρ'th candidate sk_{cand} bit-by-bit using the PSAPCE oracle to decide membership in $\mathcal{L}_{cand}$. Finally, the adversary samples a random chal-

lenge pre-image $x^* \leftarrow_\$ \{0,1\}^\lambda \setminus \{x_1, \ldots, x_{s(\lambda)+\lambda}\}$, obtain from the challenger the challenge image y^*, computes the image y' of x^* under the combiners evaluation algorithm with the secret key candidate sk_{cand} and guesses bit $b = 0$ if and only if $y' = y^*$. We illustrate the distinguisher in Figure 3.3.

Claim 3.16 *Let $\mathcal{A}$ be the adversary as described above and in Figure 3.3. $\mathcal{A}$ runs in time polynomial in the security parameter λ and wins the pseudorandomness security experiment agaginst any robust black-box DAG combiner Π_{VRF} instanciated with the candidates $\mathcal{O}_{vrf}$ and $\mathcal{O}_{up}$ as above with non-negligible advantage. Specifically*

$$\mathbf{Adv}^{\mathrm{prf}}_{\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{up}}}(\mathcal{A},\lambda) \geq \frac{1}{2} - \frac{1}{2^{\lambda+1}}$$

Proof (Claim 3.16) *Runtime*: The adversary $\mathcal{A}$ can sample $s(\lambda)+\lambda$ evaluation points, query the games evaluation oracle on them to obtain the image-proof pairs and evaluate the combiners verifiation algorithm on the point-image-proof tuples to obtain the trace in time $\mathcal{O}(s(\lambda)+\lambda)$. The set of candidate keys is bounded in size by the size of the secret key space, thus by means of binary search $\mathcal{A}$ can obtain the size with at most $s(\lambda)$ queries to the PSPACE complete oracle. With another $s(\lambda)$ queries $\mathcal{A}$ can then recover the ρ'th candidate key bit-by-bit. The final evaluation of the combiners evaluation algorithm on the candidate secret key and the challenge pre-image can also be done in polynomial time due to the efficiency requirement on the robust black-box DAG combiners. Overall, the adversary runs in time polynomial in λ and n.

Advantage: During its execution, $\mathcal{A}$ will correctly compute the size of the candidate key set, then obtain a uniformly random candidate secret key. We invoke Claim 3.14 which yields that with probability at least $\frac{1}{2} - \frac{1}{2^{\lambda+1}}$ the candidate secret key will evaluate to the same image on the challenge pre-image x^* as the games internal secret key would. Therefore, with probability at least $\frac{1}{2} - \frac{1}{2^{\lambda+1}}$ $\mathcal{A}$ wins the game. Otherwise, with probability at most $\frac{1}{2} + \frac{1}{2^{\lambda+1}}$ the adversary will not obtain the correct image y^*. If the hidden bit $b = 0$ then the adversary loses since he will guess $b = 1$, if the hidden bit is $b = 1$ then he will win except if the output $y' = y^*$ by chance. Since y is in that case sampled u.a.r. we can bound the probability of that happening with $1 - \frac{1}{|\mathcal{Y}|}$. Overall we get:

$$\mathbf{Adv}^{\mathrm{prf}}_{\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{up}}}(\lambda,\mathcal{A}) = 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathsf{VF}}(\lambda,\mathcal{A}) = 1\right] - \frac{1}{2} \right| \tag{3.36}$$

$$= |\Pr[\mathcal{A} = 1 \mid b = 0]| \tag{3.37}$$

□

Adversary $\mathcal{A}_1^{\mathsf{PSPACE},\mathcal{O}_{eval},\mathcal{O}_{vrf},\mathcal{O}_{uppc}}(\mathsf{pk})$

1 : // Obtain $\{(x_i, y_i, \pi_i, \tau_i)\}_{i\in[s(\lambda)+\lambda]}$:
2 : $\mathcal{X} \leftarrow \varnothing$
3 : **for** $i \in [s(\lambda)+\lambda]$ **do**
4 : $\quad x_i \leftarrow_{\$} \{0,1\}^{\lambda}$
5 : $\quad \mathcal{X} \leftarrow \mathcal{X} \cup \{x_i\}$
6 : $\quad (y_i, \pi_i) \leftarrow \mathcal{O}_{eval}(x_i)$
7 : $\quad \tau_i \leftarrow \mathsf{trace}(\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x_i, y_i, \pi_i))$
8 : // Obtain $|\bigcap_{i\in[s(\lambda)+\lambda]} \mathcal{SK}_{cand}(x_i, y_i, \pi_i, \tau_i)|$ using binary search:
9 : $\ell \leftarrow 0$
10 : $u \leftarrow 2^{s(\lambda)}$
11 : **while** $\ell < u$ **do**
12 : $\quad m \leftarrow \lfloor(\ell+u)/2\rfloor$
13 : $\quad b \leftarrow \mathsf{PSPACE}\big((m, \{x_i, y_i, \pi_i, \tau_i\}_{i\in[s(\lambda)+\lambda]}) \in \mathcal{L}_{size}\big)$
14 : $\quad$ **if** $b = \top$ **then**
15 : $\quad\quad u \leftarrow m$
16 : $\quad$ **else**
17 : $\quad\quad \ell \leftarrow m+1$
18 : $k \leftarrow \ell$
19 : $\rho \leftarrow_{\$} [k]$
20 : // Obtain $\mathsf{sk}_{cand} \in \mathcal{SK}_{cand}$:
21 : $\mathsf{sk}_{cand} \leftarrow \varepsilon$
22 : **for** $i \in [s(\lambda)]$ **do**
23 : $\quad b \leftarrow \mathsf{PSPACE}\big((0, \rho, i, \{x_i, y_i, \pi_i, \tau_i\}_{i\in[s(\lambda)+\lambda]}) \in \mathcal{L}_{cand}\big)$
24 : $\quad$ **if** $b = \top$ **then**
25 : $\quad\quad \mathsf{sk}_{cand} \leftarrow \mathsf{sk}_{cand}\|0$
26 : $\quad$ **else if** $b = \bot$ **then**
27 : $\quad\quad \mathsf{sk}_{cand} \leftarrow \mathsf{sk}_{cand}\|1$
28 : $x^* \leftarrow_{\$} \{0,1\}^{\lambda} \setminus \mathcal{X}$
29 : **return** $(\mathsf{sk}_{cand}, x^*)$

Adversary $\mathcal{A}_2^{\mathsf{PSPACE},\mathcal{O}_{eval},\mathcal{O}_{vrf},\mathcal{O}_{uppc}}(\mathsf{pk}, \mathsf{sk}_{cand}, x^*, y^*)$

1 : $y' \leftarrow \Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{cand}, x^*)$
2 : **return** $y' = y^*$

Figure 3.3: Adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ against $\Pi_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}$. Besides the candidate schemes $\mathcal{O}_{vrf}$ and $\mathcal{O}_{uppc}$ the oracle $\mathcal{O}_{eval}$ is the evaluation oracle from the pseudorandoness security experiment and PSPACE is a PSPACE complete oracle. We use $\|$ to denote concatenation.

Using Claim 3.16 we can now finally prove the Theorem 3.5.

Proof (Theorem 3.5) Claim 3.16 states that for every robust $(2;1)$ black-box DAG VRF-combiner Π_{VRF} there exists a polynomial time adversary $\mathcal{A}$ with non-negligible advantage against Π_{VRF} initialized with candidates $\mathcal{O}_{vrf}$ and $\mathcal{O}_{up}$. Since $\mathcal{O}_{vrf}$ implements a secure VRF we conclude that there exists no robust $(2;1)$ black-box DAG VRF combiner since there exists no polynomial time adversary $\mathcal{B}$ against $\mathcal{O}_{vrf}$. Since there exists no robust $(2;1)$ black-box DAG VRF-combiner, by Theorem 3.2, there also doesn't exist a robust $(2;1)$ black-box VRF-combiner. □

While robust $(2;1)$ black-box combiners are generally sufficient to construct robust $(n;1)$ black-box combiners, non-existence of a robust $(2;1)$ black-box combiner only allows one to deduce the impossibility of a robust $(n;\lfloor n/2 \rfloor)$ black-box combiner. This allows us to extend the result in Theorem 3.5 to an infinite family of robust black-box VRF-combiners.

Lemma 3.17 (Robust $(n;\lfloor n/2 \rfloor)$ Black-Box VRF Combiner) *There exists no robust $(n;\lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive defined in Definition 2.9 for all $n \in \mathbb{N}^+$.*

Proof (Lemma 3.17) We show how to construct a robust $(2;1)$ black-box VRF-combiner $\widetilde{\Pi}_{\mathsf{VRF}}$ from a robust $(n;\lfloor n/2 \rfloor)$ black-box VRF-combiner Π_{VRF}. The result holds by contraposition of this fact applied to Theorem 3.5. $\widetilde{\Pi}_{\mathsf{VRF}}$ is simply Π_{VRF} but the first $\lfloor n/2 \rfloor$ candidates are initialized with $\mathcal{O}_1$ and the other $\lceil n/2 \rceil$ candidates are initialized with $\mathcal{O}_2$. If either $\mathcal{O}_1$ or $\mathcal{O}_2$ implements a secure VRF then at least $\lfloor n/2 \rfloor$ candidates used to initialize Π_{VRF} are secure, the initialized combiner is secure. □

Extending this result to an impossibility of robust $(n;n-1)$ turned out to be more difficult than expected. While the proof for Theorem 3.5 doesn't seem to generalize to an impossibility result for robust $(n;n-1)$ robust black-box VRF-combiner we still believe that such constructions can be proven to be impossible. We state the result as a conjecture and defer to the appendix for a discussion on why the approaches don't easily generalize.

Conjecture 3.18 *There exists no robust $(n;n-1)$ black-box combiner in the sense of Definition 2.12 for the VRF primitive as defined in Definition 2.9 for all positive natural numbers $n \in \mathbb{N}^+$.*

Finally we want to note that the distinguisher constructed in the proof of Theorem 3.5 can predict the image of any arbitrary pre-image $x \in \{0,1\}^\lambda$. This means that the adversary is strong enough win the weak pseudorandomness and unpredictability security experiment with the same advantage as in Claim 3.16. As a corollary of Theorem 3.5, we get the impossibility of robust $(n;\lfloor n/2 \rfloor)$ black-box combiners for the VUF and WVRF primitive.

Corollary 3.19 (Robust $(n; \lfloor n/2 \rfloor)$ Black-Box VUF-combiner) *There exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the VUF primitive as defined in Definition 2.11.*

Corollary 3.20 (Robust $(n; \lfloor n/2 \rfloor)$ Black-Box WVRF-combiner) *There exists no robust $(n; \lfloor n/2 \rfloor)$ black-box combiner in the sense of Definition 2.12 for the WVRF primitive as defined in Definition 2.10.*

3.3 Relaxed Robust Black-Box VRF-Combiners

In the previous section we established an impossibility of robust $(n; \lfloor n/2 \rfloor)$ black-box combiners for the VRF, WVRF and VUF primitives and conjectured that the result extends to robust $(n; n-1)$ black-box combiners. A natural follow-up question is whether the impossibility results can be circumvented by relaxing by either strengthening by introducing additional assumptions on the candidates. In this section, we show that under the additional assumption that all candidates satisfy unique provability and perfect correctness, there exist simple robust $(n; 1)$ black-box combiners for VRFs.

We argue that the additional assumptions are reasonable in practice. Since perfect correctness and unique provability are information theoretic properties they don't rely on computational hardness assumptions but can generally be proved rigorously. As a result, for concrete VRF constructions that satisfy those properties, the guarantee holds regardless of advances in cryptography and therefore there is no reason to hedge against them by using combiners. Examples are ECVRF [**?**] and LatticeVRF [**?**].

As already mentioned in Section 1.2, PRFs admit a simple robust $(n; 1)$ black-box combiner which simply xors the outputs of all PRF candidates. We show that this construction can be extended to VRFs by simply computing the transcript of the evaluation algorithms and providing this as a proof.

Lemma 3.21 (Relaxed Robust $(n; 1)$ Black-Box VRF Combiners) *There exist robust $(n; 1)$ black-box combiners in the sense of Definition 2.12 for the VRF primitive Definition 2.9 if we assume that all candidates are uniquely provable in the sense of Definition 2.5 and perfectly correct in the sense of Definition 2.4.*

We construct a robust $(2; 1)$ black-box VRF-combiner under the assumption that all candidates are perfectly correct and uniquely provable. A robust $(n; 1)$ black-box VRF-combiner can then be constructed by iteratively applying the construction, combining the previous combiner's output with the next candidate oracle until all n candidate schemes are incorporated.

The construction, illustrated in Figure 3.4, closely follows the construction of the robust $(2; 1)$ black-box PRF-combiner in Section 1.2. The key generation algorithm samples a key pair per candidate and combines the secret and public keys to form the combined key pair. The evaluation algorithm evaluates the point x with both candidate schemes to obtain (y_1, π_1) and (y_2, π_2). The image is then $y = y_1 \oplus y_2$ with the proof being $\pi = (y_1, \pi_1, y_2, \pi_2)$. The verification algorithm works by first checking whether both values y_1, y_2 with proof π_1, π_2 in the proof verify under the verification algorithm of the respective candidate and secondly checks whether $y = y_1 \oplus y_2$.

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$

1 : $(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow_\$ \mathcal{O}_1.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
2 : $(\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow_\$ \mathcal{O}_2.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : **return** $((\mathsf{sk}_1, \mathsf{sk}_2), (\mathsf{pk}_1, \mathsf{pk}_2))$

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

Algorithm $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : $b_1 \leftarrow \mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1)$
4 : $b_2 \leftarrow \mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2)$
5 : **if** $y = y_1 \oplus y_2 \wedge b_1 = 1 \wedge b_2 = 1$ **then**
6 : **return** 1
7 : **else**
8 : **return** 0

Figure 3.4: Relaxed robsut $(2;1)$ black-box VRF-combiner Π_{VRF} for uniquely provable and perfectly correct candidates $\mathcal{O}_1, \mathcal{O}_2$.

Claim 3.22 *The combiner in Figure 3.4 is perfectly correct in the sense of Definition 2.4 and uniquely provable in the sense of Definition 2.5 if all candidates are perfectly correct Definition 2.4 and uniquely provable Definition 2.5.*

The proof of this claim follows essentially by definition of the combiner construction and the assumed unique provability and pseudorandomness of all candidates. We believe that its easy to see that the claim holds and therefore defer the proof to Chapter 4.

from here

Claim 3.23 *The combiner in Figure 3.4 is $(\mathsf{t}_{comb}^{prf}, \varepsilon_{comb}^{prf})$-pseudorandom in the sense of Definition 2.6 if at least one candidate are $(\mathsf{t}_i^{prf}, \varepsilon_i^{prf})$-pseudorandom. In particular if at least one candidate is pseudorandom then the combiner is pseudorandom.*

Assume, towards a contradiction, that there exists an adversary $\mathcal{A}$ running in time t that breaks pseudorandomness of the combiner $\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}$ with advantage ε, we show that we can construct an adversary $\mathcal{R}_i$ running in time t_i that wins the prf security experiment of the i'th candidate $\mathcal{O}_i$ with advantage ε_i for $i \in [2]$. The reduction $\mathcal{R}_i$ samples it own secret key for candidate $\mathcal{O}_{2-i}$ and uses $\mathcal{O}_{eval}$ for candidate $\mathcal{O}_i$ simulate the combiner in the prf security experiment for the adversary $\mathcal{A}$. $\mathcal{A}$ then makes a prediction for the hidden

bit. If the hidden bit of the challenger is 0 then $\mathcal{A}$ is simulated perfectly in the sense that the queries are answered exactly as they would have been answered in the combiners prf security experiment. In that case $\mathbb{R}_i$ wins the experiment if and only if $\mathcal{A}$ wins. If $b = 1$ the output of $\mathcal{O}_{eval}$ is a uniformly random element in the image co-domain and therefore the queries made by $\mathcal{A}$ are also answered with a uniformly random element in the co-domain. Also in this case $\mathcal{A}$ is simulated perfectly. and uses adversary $\mathcal{A}$ to break that one candidate. We show the reduction $\mathcal{R}_i$ in Figure 3.5.

Reduction $\mathcal{R}_i^{\mathcal{O}_{i_{eval}}}(\mathsf{pk}_i)$

1 : $b \leftarrow_\$ \{0,1\}$
2 : $(\mathsf{sk}_{2-i}, \mathsf{pk}_{2-i}) \leftarrow_\$ \mathcal{O}_{eval,2-i}.\mathsf{Gen}_{\mathsf{VRF}}(1^\lambda)$
3 : $\mathsf{sk} \leftarrow (\mathsf{sk}_1, \mathsf{sk}_2)$
4 : $\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$
5 : $S \leftarrow \varnothing$
6 : $(\sigma, x^*) \leftarrow_\$ \mathcal{A}^{\mathcal{O}_{comb_{eval}}}(\mathsf{pk}_1, \mathsf{pk}_2)$
7 : $y_1 \leftarrow_\$ \mathcal{Y}$
8 : $b' \leftarrow_\$ \mathcal{A}(1^\lambda, x^*, \sigma, y_b)$
9 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{comb_{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y_i, \pi_i) \leftarrow \mathcal{O}_{i_{eval}}(x)$
3 : $(y_{2-i}, \pi_{2-i}) \leftarrow \mathcal{O}_{2-i}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_{2-i}, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

Figure 3.5: Reduction.

The reduction $\mathcal{R}_i$ makes as many queries to its evaluation oracle $\mathcal{O}_{i_{eval}}$ as $\mathcal{A}$ does to its evaluation oracle $\mathcal{O}_{comb_{eval}}$ and makes additionally as many queries to the oracle $\mathcal{O}_{2-i}$. Besides the oracle queries every computation performed by $\mathcal{R}_i$ is efficiently computable. Since we assume $\mathcal{A}$ to be efficient (ppt.) the reduction $\mathcal{R}_i$ msut also be efficient (ppt.).

The advantage of the reduction $\mathcal{R}_i$ in the prf security experiment against candidate $\mathcal{O}_i$ can be bounded from below as follows.

$$\mathbf{Adv}_{\mathsf{i}}^{\mathrm{prf}}(\lambda, \mathcal{R}_i) = \tag{3.38}$$

finish the reduction

Chapter 4

Conclusion

We introduced robust black-box DAG combiners that make the structure of robust black-box combiners more explicit while capturing the same set of constructions. Subsequently we answered the question whether there exist robust $(2;1)$ black-box VRF-combiner negatively. The result implies an impossibility result on the family of robust $(n;\lfloor n/2 \rfloor)$ black-box VRF-combiners placing a lower bound on the number of secure instances needed to combine VRFs in a robust way.

On the other hand we show that if we strengthen the assumptions on the candidate schemes, specifically by requiring all of them to be uniquely provable and perfectly correct, then VRF admit simple combiner similar to the known PRF combiners.

Extensions. While we strongly believe robust $(n;n-1)$ black-box VRF-combiner to be impossible, extending the proof for the impossibility of robust $(2;1)$ black-box VRF-combiners to the more general case seems difficult. We pose this question as a conjecture.

Additionally, it is natural to think about weaker relaxations of robust black-box VRF-combiners that circumvent the impossibility result than the one we presented. For instance, it is interesting to study the possibility of combining VRFs if the combiner only needs to satisfy statistical correctness.

Finally, even for the strong relaxation of VRF-combiners that admit simple XOR combiner it is intersting to study relaxed robust black-box combiners for more advanced notions of VRFs such as constrainable VRFs [**?**], key-homomorphic VRFs [**?**], aggregable VRFs [**?**] and unbiasable VRFs [**?**].

Bibliography

[BCK96] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. pages 1–15, 1996.

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, October 1986.

[KL14] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Chapman and Hall, CRC Press, third edition, 2014.

[Kra10] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. pages 631–648, 2010.

[MRV99] Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. Verifiable random functions. pages 120–130, 1999.

Appendix

Proof of Lemma .1

We present the proof of the difference lemma, formally:

Lemma .1 (Difference Lemma) *Let $\mathcal{W}_0, \mathcal{W}_1, \mathcal{E}$ be events with*

$$\Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] = \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]$$

Then we can bound the difference in probability of $\mathcal{W}_0$ and $\mathcal{W}_1$ with

$$|\Pr[\mathcal{W}_0] - \Pr[\mathcal{W}_1]| \leq \Pr[\mathcal{E}]$$

Proof of Lemma .1. Let $\mathcal{W}_0, \mathcal{W}_1, \mathcal{E}$ be events and assume $\Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] = \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]$. Then we can deduce the following:

$$\begin{aligned}
&|\Pr[\mathcal{W}_0] - \Pr[\mathcal{W}_1]| && (.1)\\
=&|\Pr[\mathcal{W}_0 \wedge \mathcal{E}] + \Pr[\mathcal{W}_0 \wedge \neg\mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \neg\mathcal{E}]| && (.2)\\
=&|\Pr[\mathcal{W}_0 \wedge \mathcal{E}] - \Pr[\mathcal{W}_1 \wedge \mathcal{E}]| && (.3)\\
=&|\Pr[\mathcal{W}_0 \mid \mathcal{E}]\Pr[\mathcal{E}] - \Pr[\mathcal{W}_1 \mid \mathcal{E}]\Pr[\mathcal{E}]| && (.4)\\
=&|\Pr[\mathcal{W}_0 \mid \mathcal{E}] - \Pr[\mathcal{W}_1 \mid \mathcal{E}]|\Pr[\mathcal{E}] && (.5)\\
\leq&\Pr[\mathcal{E}] && (.6)
\end{aligned}$$

Conjecture

We quickly want to make a few remarks towards the idea of extending the proof of **??** to the more general case of **??**. The direct proof fails because of the subtle reason that the hidden oracle queries need to be candidate-wide irrelevant but not jointly irrelevant over queries to many distinct candidate schemes. In the proof of **??** we argue that for oracle $\mathcal{O}_2$ all hidden oracle queries, i.e. all oracle queries not contained in the transcript of the combiners verification algorithm are exactly the queries that would influence the combiner output if changes. On the other hand, for any arbitrary assignment of

the hidden queries, the output of the combiner stays constant as long as the non-hidden queries and the queries to $\mathcal{O}_1$ are honestly answered.

This idea gives rise to the attack of iteratively filtering the secret key space on a polynomial number of points by keeping in every round (1.) at least the true secret key, (2.) only secret keys for which the output of the combiner is the same as for the true secret key. We argue that after a sufficient number of filtering steps, any remaining candidate has sufficiently high chance of evaluating to the same output as the combiner does with the true secret key on a uniformly random evaluation point.

However, since the hidden queries must not necessarily be jointly-irrelevant this argument cannot be applied if we have more candidates, of which still $n - 1$ need to be secure for the combiner to be required to implement unique provability and pseudorandomness.

TODO example of axis-constant function

Proof of Claim 3.22

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).